

UNIVERSITÀ POLITECNICA DELLE MARCHE
INGEGNERIA INFORMATICA E DELL'AUTOMAZIONE

Controllo di trazione e sterzata di un veicolo in scala 1:10 mediante radiocomando



Corso di
LABORATORIO DI AUTOMAZIONE
Anno accademico 2025-2026

Studenti:

Tarek Naja
Marco Sambughi
Sara Vaccaro

Professore:

Andrea Bonci

Dottorando:

Alessandro Di Biase



Dipartimento di Ingegneria dell'Informazione

Indice

Introduzione	2
1 Descrizione del sistema	4
1.1 Controllo veicolo	5
1.1.1 Controllo della trazione	5
1.1.2 Controllo dello sterzo	7
2 Hardware	8
2.1 Telaio e meccaniche di supporto	8
2.2 Nucleo-144 STM32H755zi-q	9
2.3 Motore DC Reely 550er	9
2.4 Driver Motore Cytron MD10C R3	10
2.5 Encoder incrementale CUI Devices AMT102	11
2.6 Servomotore Miuzei MS24	11
2.7 Alimentazione	12
2.7.1 Convertitori DC-DC	12
2.7.2 Batteria ai polimeri di litio	12
2.8 Inertial Measurement Unit	13
2.9 Radiocomando e Ricevitore Microzone	13
2.9.1 Funzionamento	14
2.9.2 Canali del ricevitore	15
2.10 Schema dei collegamenti	16
3 Software	17
3.0.1 STM32CubeIDE	17
3.0.2 MATLAB	17
3.0.3 Acquisizione dati via Terminale	17
4 Implementazione del firmware	19
4.1 Diagramma di flusso	19
4.2 Implementazione relativa al radiocomando	20
4.2.1 Impostazioni iniziali timers e calcolo preliminare di frequenza e duty	20
4.2.2 Controllo della trazione	28
4.3 Implementazione del Controllo dello Sterzo	33
4.3.1 servo_motor.c	38
5 Test e risultati	42
5.1 Trazione	42
5.2 Sterzo	47
5.2.1 Variazione velocità lineare	51
5.2.2 Variazione velocità angolare	52

Introduzione

Il presente elaborato documenta le fasi di progettazione, sviluppo e validazione del firmware di controllo per un veicolo elettrico in scala 1:10. Nello specifico, il lavoro operativo è stato suddiviso in tre task fondamentali: l'interfacciamento e la decodifica dei segnali radio, la regolazione della trazione per la gestione della velocità e il controllo direzionale dello sterzo, questi ultimi implementati mediante algoritmi di controllo PID. Nel primo capitolo, viene fornita una descrizione generale del sistema, definendo il contesto operativo e le logiche alla base del controllo della trazione e dello sterzo. Successivamente, il secondo e il terzo capitolo offrono una panoramica dettagliata dell'architettura hardware e degli ambienti software impiegati, delineando tutti i requisiti strumentali necessari per la riproduzione del setup sperimentale. Al suo interno viene dapprima analizzata la struttura generale del codice, per poi scendere nel dettaglio degli algoritmi sviluppati per l'acquisizione e l'elaborazione dei segnali del radiocomando, illustrando l'impostazione dei timer, il calcolo del Duty Cycle e la gestione degli interrupt. A partire da queste fondamenta, vengono discussi i codici e le librerie specifiche implementate mediante l'utilizzo di regolatori PID, analizzando dapprima l'erogazione della trazione tramite la lettura dell'encoder, e successivamente il controllo direzionale dello sterzo, inclusa la sua necessaria calibrazione meccanica. Per dimostrare la validità del lavoro svolto, ciascuna di queste tre macro-fasi di implementazione software, è corredata da un'analisi dedicata ai test sul campo e ai risultati ottenuti, permettendo di verificare concretamente la prontezza e la stabilità della risposta dinamica del veicolo. Infine, la relazione si conclude con una panoramica sui possibili sviluppi futuri, delineando potenziali miglioramenti e ottimizzazioni per le successive evoluzioni del prototipo.

1 Descrizione del sistema

La seguente relazione si basa sull'analisi dell'architettura di comunicazione via radio tra radiocomando e microcontrollore. Nel dettaglio, il trasmettitore impiega onde elettromagnetiche per l'invio dei pacchetti dati, sfruttando la modulazione di un'onda portante che codifica le informazioni variando parametri dell'onda stessa, quali l'ampiezza, la frequenza o la fase. L'espressione matematica dell'onda portante è definita dalla seguente funzione:

$$s(t) = A \cos(2\pi f_c t + \phi) \quad (1.1)$$

dove:

- A : ampiezza dell'onda;
- f_c : frequenza portante;
- t : tempo;
- ϕ : fase iniziale.

L'azione dell'utente sui comandi del radiocomando genera una sequenza di bit, segnale elettrico, che l'antenna del trasmettitore provvede a convertire in onda elettromagnetica, irradiandola nello spazio a una determinata frequenza f . Il modulo ricevitore, essendo sintonizzato sulla stessa frequenza, intercetta il segnale e lo riconverte nel formato elettrico. Questi dati vengono poi inoltrati al microcontrollore, il quale ha il compito di interpretare le istruzioni e comandare l'hardware per eseguire determinate azioni. Per la traduzione pratica dei comandi, il microcontrollore impiega la modulazione a larghezza di impulso, PWM. Il parametro fondamentale per questa tecnica è il *Duty Cycle*, calcolato come segue:

$$\text{Duty Cycle (\%)} = \frac{T_{on}}{T_{on} + T_{off}} \times 100 \quad (1.2)$$

dove T_{on} è la durata dell'impulso "alto", T_{off} la durata dell'impulso "basso" e la loro somma definisce il periodo totale dell'onda. Per quanto riguarda l'analisi della traiettoria del veicolo, si adotta il modello cinematico della bicicletta, approssimando la dinamica del veicolo a quattro ruote riducendola a due. Le equazioni cinematiche descrivono il moto attraverso le variabili di posizione (x, y) e l'angolo di orientamento della vettura. Considerando costante la velocità v del veicolo, l'assenza di slittamento e che l'intera rotazione del veicolo avvenga attorno al CIR (Centro Istantaneo di Rotazione), il modello viene descritto dal seguente sistema:

$$\begin{cases} \dot{x}(t) = v \cos(\theta) \\ \dot{y}(t) = v \sin(\theta) \\ \dot{\theta}(t) = \frac{v}{L} \tan(\delta) \end{cases} \quad (1.3)$$

I parametri fondamentali del modello sono:

- L : distanza tra asse anteriore e posteriore (passo);
- δ : angolo di sterzata (ruote anteriori);
- θ : angolo di imbardata del veicolo;
- v : velocità della macchina;
- $\dot{x}, \dot{y}$: velocità nei due assi (derivate della posizione rispetto al tempo).

Nel caso in cui si considerasse lo slittamento si passerebbe ad un modello dinamico e la velocità di rotazione non dipenderebbe più solo dalla geometria. Il modello descritto quindi è una semplificazione del modello reale, ma risulta uno strumento efficace per comprendere la cinematica di base del veicolo e la traiettoria in funzione dell'angolo di sterzo, sfruttando unicamente due assi.

1.1 Controllo veicolo

Nelle prossime sezioni verranno analizzate, dal punto di vista teorico, le tecniche di controllo utilizzate per la gestione degli attuatori presenti sul veicolo.

1.1.1 Controllo della trazione

Per il controllo della trazione, ovvero la regolazione della velocità longitudinale del veicolo, si è optato per un sistema di controllo in retroazione. Tale architettura risulta fondamentale non solo per garantire il preciso inseguimento della velocità di riferimento, ma anche per compensare i disturbi esterni non prevedibili, quali le variazioni di attrito del terreno, le pendenze o le cadute di tensione della batteria durante l'erogazione di corrente. Il sistema misura costantemente la velocità reale del veicolo e la confronta con la velocità di riferimento, generando un segnale di errore $e(t)$ da minimizzare:

$$e(t) = v_{\text{target}} - v_{\text{reale}} \quad (1.4)$$

Il feedback di velocità è fornito da un encoder incrementale calettato sull'albero motore: il sensore genera una serie di impulsi che vengono conteggiati dal microcontrollore all'interno di un periodo di campionamento a tempo fisso. Dalla derivata discreta di questi conteggi si ricava l'effettiva velocità di rotazione del rotore. Per minimizzare l'errore $e(t)$ e calcolare lo sforzo di controllo da applicare al motore, tradotto poi nel *Duty Cycle* del segnale PWM, viene impiegato un controllore PID (Proporzionale-Integrale-Derivativo). La legge di controllo nel dominio del tempo continuo è descritta dalla seguente equazione:

$$u(t) = K_p \cdot e(t) + K_i \int e(t) dt + K_d \frac{de(t)}{dt} \quad (1.5)$$

dove $u(t)$ rappresenta l'azione di comando generata dal controllore, mentre K_p , K_i e K_d sono rispettivamente le costanti di guadagno dell'azione proporzionale, integrale e derivativa. Nello specifico:

- **Guadagno proporzionale (K_p):** fornisce un'uscita di controllo direttamente proporzionale all'errore istantaneo. Aumentare il guadagno K_p rende il sistema più reattivo e riduce il tempo di salita. Tuttavia, da sola non è sufficiente a garantire il perfetto inseguimento del riferimento: un'azione puramente proporzionale lascia un errore a regime e, se eccessiva, induce forti oscillazioni e instabilità.
- **Guadagno integrale (K_i):** somma l'errore nel tempo, tenendo conto della “storia” del sistema. Il suo scopo fondamentale è annullare completamente l'errore a regime. Tuttavia, un accumulo prolungato di questo termine, ad esempio durante la fase di avvio o nel caso in cui le ruote slittino senza far muovere il veicolo, porta a un eccessivo incremento dell'azione di controllo. Questo si traduce in partenze improvvise o risposte sproporzionate non appena il target cambia, problematica che nel software è stata mitigata azzerando il termine integrale in specifiche condizioni di sicurezza.
- **Guadagno derivativo (K_d):** reagisce alla velocità di variazione dell'errore, anticipandone l'andamento futuro. Funge essenzialmente da “smorzatore”, con lo scopo di contrastare le eventuali oscillazioni introdotte dalle componenti precedenti. Tuttavia, per il controllo longitudinale di veicoli di questa scala, l'azione derivativa risulta spesso superflua o persino controproducente, in quanto tende ad amplificare matematicamente le normali micro-vibrazioni meccaniche o i piccoli scatti letti dall'encoder. Per questo motivo, è prassi comune nella robotica mobile di base porre direttamente $K_d = 0$, semplificando l'architettura a un più robusto e stabile controllore PI.

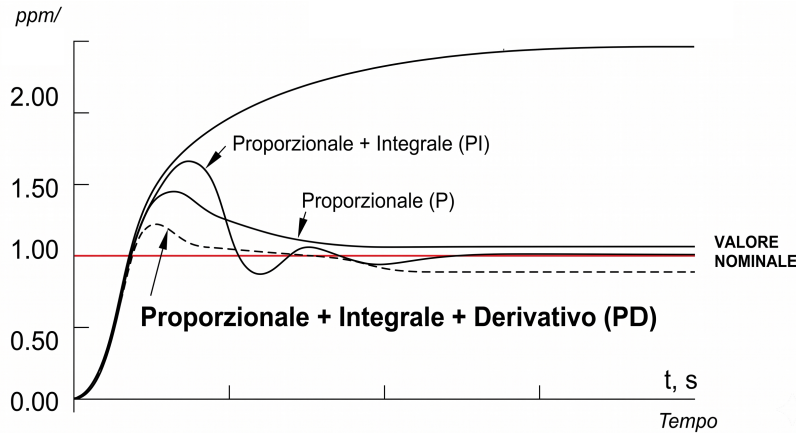


Figure 1.1

La determinazione dei parametri ottimali (K_p , K_i , K_d) prende il nome di calibrazione o *tuning*. Per la dinamica del veicolo in esame, tale procedura è stata condotta mediante calibrazione empirica in laboratorio. Partendo da valori conservativi e analizzando visivamente e tramite telemetria la risposta del motore alle variazioni di velocità richieste dal radiocomando, i guadagni sono stati ottimizzati per garantire l'inseguimento del target senza innescare instabilità.

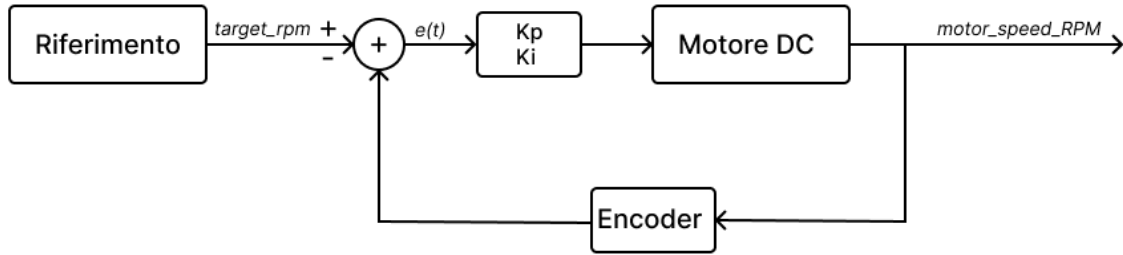


Figure 1.2

1.1.2 Controllo dello sterzo

La gestione direzionale del veicolo è affidata al sistema di sterzo delle ruote anteriori e ai segnali inviati dal radiocomando. L'attuazione meccanica è realizzata mediante un servomotore, un tipo particolare di motore che, agendo sui braccetti dello sterzo, è in grado di ruotare in una specifica posizione e mantenerla. Inoltre, in questo prototipo è stato implementato un controllo a ciclo chiuso per garantire la massima precisione nell'inseguimento della traiettoria e compensare i disturbi dinamici come gli attriti del terreno. Di conseguenza, il controllo prevede l'utilizzo di un regolatore PID, implementato all'interno del microcontrollore STM32. Il controllore riceve in ingresso l'errore $e(t)$, definito come la differenza tra l'angolo di sterzo di riferimento richiesto $\delta_{\text{ref}}(t)$, letto dal radiocomando, e l'angolo reale misurato dai sensori di bordo $\delta_{\text{reale}}(t)$:

$$e(t) = \delta_{\text{ref}}(t) - \delta_{\text{reale}}(t) \quad (1.6)$$

L'azione correttiva calcolata dall'algoritmo PID viene tradotta in un segnale di pilotaggio PWM inviato al servomotore. Per questa tipologia di attuatori, il controllo della posizione non dipende dal *Duty Cycle* percentuale classico, bensì dalla larghezza assoluta dell'impulso positivo T_{on} all'interno di un periodo fisso di 20 ms, corrispondente ad una frequenza di 50 Hz.

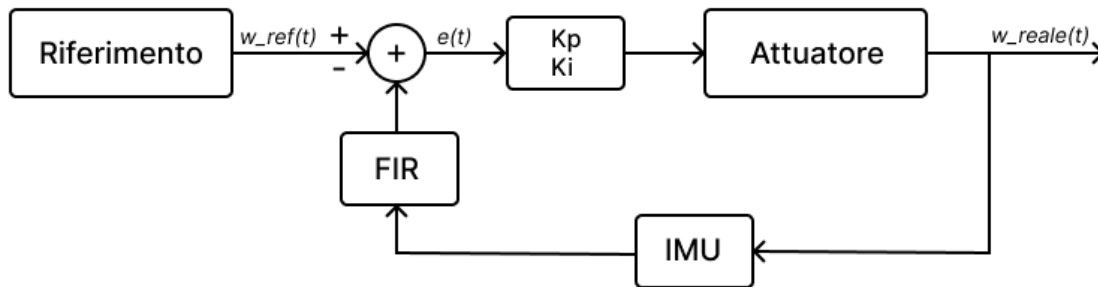


Figure 1.3

2 Hardware

Nel presente capitolo viene analizzata in dettaglio l'architettura hardware del prototipo, descrivendo le specifiche tecniche e il ruolo funzionale di ciascun componente. Verranno presi in analisi i seguenti elementi costitutivi:

- Telaio e meccanica di supporto;
- Microcontrollore Nucleo-144 STM32H755zi-q;
- Motore DC Reely 550er;
- Driver Motore Cytron MD10C R3;
- Encoder incrementale CUI Devices AMT102;
- Servomotore Miuzei MS24;
- Sistema di Alimentazione e Power Distribution Board (PDB);
- Batteria ai polimeri di litio (LiPo) 2S (2 celle);
- Unità di Inerzia Magneto-Inerziale (IMU) Bosch BNO055;
- Radiocomando e Ricevitore Microzone.

2.1 Telaio e meccaniche di supporto

La struttura portante e la cinematica del veicolo sono basate su uno chassis in scala 1:10 derivato dal settore del modellismo dinamico. Per rendere lo chassis idoneo agli scopi del progetto, la struttura originale è stata sottoposta a una serie di interventi di personalizzazione. In particolare, è stata progettata e installata una piastra di supporto superiore per incrementare la superficie utile a bordo, permettendo l'alloggiamento di tutta la componentistica elettronica.



Figure 2.1

2.2 Nucleo-144 STM32H755zi-q

L'unità di elaborazione centrale e di controllo del veicolo è costituita dalla scheda di sviluppo Nucleo-144 dotata del microcontrollore ad altissime prestazioni STM32H755ZIT6Q di STMicroelectronics. L'elemento distintivo di questo dispositivo è la sua architettura hardware *dual-core* eterogenea, che mette a disposizione due processori indipendenti a bordo dello stesso chip:

- **Core ARM Cortex-M7 (32-bit):** operante a una frequenza massima di 480 MHz.
- **Core ARM Cortex-M4 (32-bit):** operante a una frequenza massima di 240 MHz.

L'interazione e lo scambio di dati ad alta velocità tra i due core avvengono in modo sincrono tramite un'interfaccia di comunicazione interna basata su memoria condivisa e semafori hardware. La scheda offre un ricco set di periferiche hardware dedicate che risultano fondamentali per gli obiettivi del progetto: i timer avanzati per la generazione dei segnali PWM di trazione e sterzo, i timer configurati in modalità *Encoder Interface* per la lettura diretta in quadratura dell'encoder AMT102, e i bus di comunicazione analogici e digitali, come I²C o UART, per l'interfacciamento con i sensori di bordo. La configurazione delle periferiche e lo sviluppo del codice in linguaggio C sono stati interamente gestiti tramite l'ecosistema software ufficiale di STMicroelectronics, utilizzando lo strumento di generazione grafica STM32CubeMX per l'inizializzazione dell'hardware e l'ambiente di sviluppo integrato STM32CubeIDE per la scrittura, compilazione e il *debug* del *firmware*.

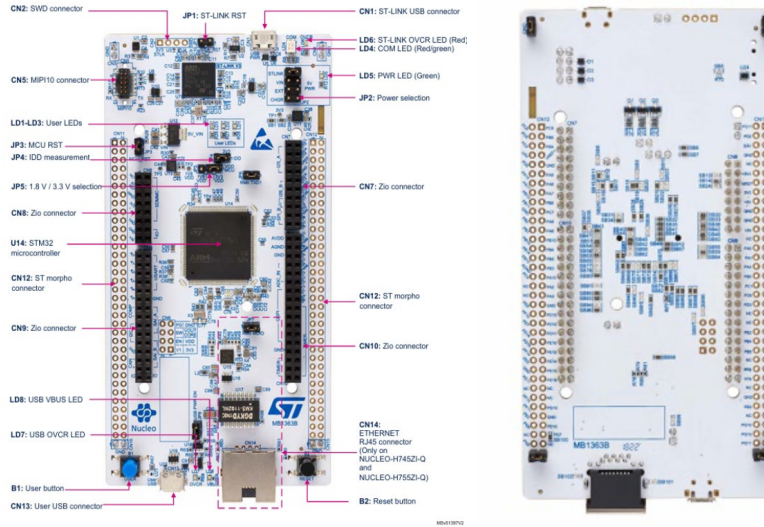


Figure 2.2

2.3 Motore DC Reely 550er

La trazione del veicolo viene garantita da un motore collegato alla trasmissione dello chassis.



Figure 2.3

In particolare, viene utilizzato un motore DC a collettore con le seguenti caratteristiche:

Parametro	Valore
Tensione nominale	4.8 – 12 VDC
Velocità di rotazione	1000 – 18000 RPM
Coppia	100 – 900 g·cm
Diametro albero	3.175 mm

2.4 Driver Motore Cytron MD10C R3

Il motore viene controllato tramite modulazione PWM attraverso il driver MD10C-R3. In particolare, il controllo del motore attraverso questo driver avviene tramite due segnali:

- **DIR:** controllo del verso di rotazione (Segnale digitale alto/basso)
- **PWM:** controllo della velocità di rotazione (segnale PWM)



Figure 2.4

2.5 Encoder incrementale CUI Devices AMT102

Inoltre, per effettuare il controllo del motore in velocità è necessario avere un feedback sulla velocità del motore stesso: ciò è stato reso possibile montando un encoder sull'asse posteriore del motore. In particolare, è stato utilizzato un encoder rotativo incrementale modello AMT102.



Figure 2.5

2.6 Servomotore Miuzei MS24

Per operare il meccanismo di sterzo del veicolo è stato impiegato un servo motore in grado di regolare il movimento angolare in modo molto preciso e proprio per questo viene comunemente utilizzato in diverse apparecchiature elettroniche e applicazioni che richiedono un controllo di posizione accurato.



Figure 2.6

Il dispositivo è controllato mediante un segnale PWM fornito in ingresso, calcolato dagli algoritmi di controllo implementati sul microcontrollore.

2.7 Alimentazione

Il sistema è composto da numerose componenti, che non operano tutte alla stessa tensione.

2.7.1 Convertitori DC-DC

Per questo motivo è stato necessario utilizzare dei convertitori DC-DC XL4015, così da garantire le alimentazioni necessarie.



Figure 2.7

2.7.2 Batteria ai polimeri di litio

Il sistema è alimentato da una batteria ai polimeri di litio (LiPo) 2S (2 celle) con una tensione nominale di 7.4V e una capacità di 5500 mAh, in grado di erogare una corrente di 5.5A per la durata di un'ora, garantendo una corretta alimentazione avendo una discreta autonomia.



Figure 2.8

2.8 Inertial Measurement Unit

L'IMU è un dispositivo che misura l'accelerazione, la velocità angolare e talvolta anche il campo magnetico. Solitamente integra un accelerometro, un giroscopio e un magnetometro, ognuno avente 3 assi (in questa configurazione l'IMU si dice a 9 gradi di libertà, 9DOF). Tali misure sono di fondamentale importanza per applicazioni in cui è richiesta la navigazione e il controllo di veicoli o in altri dispositivi mobili. L'IMU scelto per questa applicazione è il BNO055 prodotto dalla Bosch Sensortech. È caratterizzato dall'inclusione di un microcontrollere a 32 bit con algoritmi di sensor fusion integrati, che combinano i dati grezzi dei sensori per fornire informazioni precise su orientamento, accelerazione e rotazione dell'unità. Il BNO055 può comunicare tramite UART o I2C.

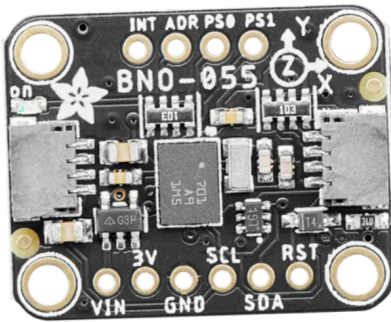


Figure 2.9

2.9 Radiocomando e Ricevitore Microzone

Il radiocomando utilizzato per inviare segnali PWM al fine di controllare la macchinina con un remote controller è il “Microzone 6CH Remote Transmitter Receiver System for Drones/Quadcopters” modello: “MC6C”.



Figure 2.10

Il radiocomando dispone di sei canali e la banda va dai 2,401 GHz a 2,479 GHz. La distanza massima di controllo è di 800m e utilizza 4 batterie xAA. Il radiocomando è inoltre dotato di ricevitore e trasmettitore che hanno le seguenti caratteristiche:

Table 2.1: *Caratteristiche del trasmettitore*

Caratteristiche	Valori
Potenza di trasmissione	≤ 70 mW
Range per il controllo	800 m
Tensione (DC)	+6 V
Peso	550 g
Modalità di modulazione	FHSS

Table 2.2: *Caratteristiche del ricevitore*

Caratteristiche	Valori
Banda di frequenza	2.400 – 2.483 GHz
Distanza lineare	> 800 m
Tensione di alimentazione (DC)	4,5 – 6 V
Tipo di segnale	PWM / SBUS
Antenna	Integrata nel ricevitore
Dimensioni	$45 \times 45 \times 10$ mm
Peso	9,6 g

2.9.1 Funzionamento

Il trasmettitore e il ricevitore comunicano tramite onde radio, le quali fungono da vettore per le istruzioni di comando generate dal pilota. Il modulo ricevitore svolge un ruolo cruciale in questa architettura: elabora i pacchetti radio in ingresso e genera in uscita un segnale PWM indipendente per ciascun canale. La variazione del *Duty Cycle* di questi segnali traduce proporzionalmente i movimenti fisici eseguiti sul radiocomando in dati quantificabili dal microcontrollore.



Figure 2.11

In fase di configurazione, la gestione della dinamica del veicolo è stata mappata in modo strategico su specifiche levette:

- **Controllo trazione (cerchio verde):** La leva di destra è dedicata alla velocità longitudinale. Il suo azionamento lungo l'asse verticale permette di regolare la potenza erogata e far avanzare o retrocedere il veicolo.
- **Controllo sterzo (cerchio giallo):** La leva di sinistra è deputata alla gestione dell'angolo di sterzata. Questa specifica scelta di *mapping* ergonomico è stata fatta perché spingere la leva orizzontalmente (a destra o a sinistra) emula in modo molto più naturale e intuitivo il movimento direzionale delle ruote. Tale spostamento trasversale si traduce direttamente nella variazione del segnale PWM inviato al servomotore.
- **Armamento motore (cerchio blu):** Lo *switch* a tre scatti, situato in alto a sinistra, funge da sistema di sicurezza (abilitazione hardware) per l'armamento del motore. Nelle posizioni estreme (completamente in alto o in basso) il motore è forzatamente spento e disarmato, mentre posizionando la levetta al centro il sistema viene armato ed è pronto a ricevere potenza dalla trazione.
- **Regolazione fine (cerchi rossi):** Infine, i *trimmer* fisici consentono di applicare un *offset* ai segnali. Questi pulsanti traslano leggermente l'intervallo di base del *Duty Cycle* sui primi quattro canali, rivelandosi essenziali per calibrare elettronicamente lo "zero" meccanico dello sterzo e del motore senza dover intervenire via software.

2.9.2 Canali del ricevitore

Il ricevitore è dotato di sei canali che permettono di gestire i segnali generati dallo spostamento delle varie levette. La seguente tabella riporta la relazione tra canali e leve.

Table 2.3: *Mappatura dei canali del radiocomando*

Canale	Leva del radiocomando
Channel 1	Leva destra orizzontale
Channel 2	Leva destra verticale
Channel 3	Leva sinistra verticale
Channel 4	Leva sinistra orizzontale
Channel 5	Levetta in alto a sinistra (a 3 posizioni)
Channel 6	Rotella in alto a destra

Per la realizzazione del progetto sono stati utilizzati solo i canali 2 e 4 per le levette.

2.10 Schema dei collegamenti

La Figura 2.12 illustra lo schema generale dei collegamenti elettrici e di segnale che costituiscono l'architettura hardware del prototipo. Il sistema è centralizzato attorno alla scheda di sviluppo STM32, la quale funge da nodo principale per l'elaborazione logica, l'acquisizione dei dati e il controllo degli attuatori.

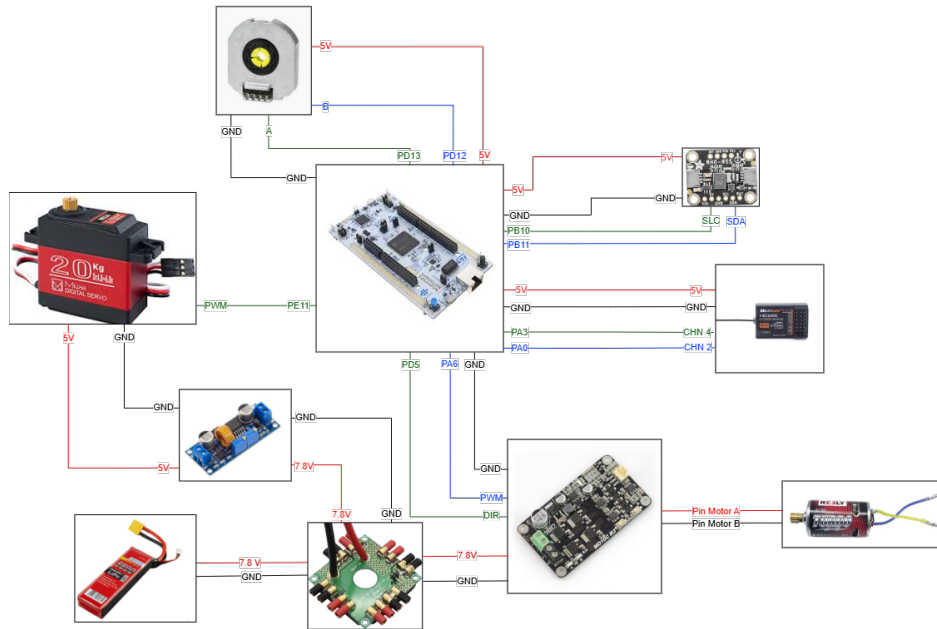


Figure 2.12: *Schema dei collegamenti*

3 Software

Il presente capitolo descrive gli applicativi software impiegati per lo sviluppo del progetto. L'utilizzo di tali strumenti ha consentito la programmazione del microcontrollore e la successiva implementazione degli algoritmi di controllo necessari al funzionamento dinamico del veicolo. Inoltre, l'infrastruttura software si è rivelata un supporto fondamentale durante la fase di testing, permettendo l'acquisizione e la visualizzazione in tempo reale dei parametri di telemetria e dei grafici di risposta del sistema.

3.0.1 STM32CubeIDE

Il firmware per la scheda STM32H755ZI-Q è stato interamente sviluppato in linguaggio C avvalendosi di STM32CubeIDE, l'ambiente di sviluppo integrato ufficiale di STMicroelectronics. Quest'ultimo mette a disposizione un'avanzata interfaccia grafica che, tramite un file di configurazione con estensione `.ioc`, permette di inizializzare l'hardware del microcontrollore in modo rapido e intuitivo. Tra le principali impostazioni gestite figurano:

- Configurazione dell'albero di clock del sistema;
- Assegnazione e configurazione dei pin di input/output (GPIO);
- Setup dei Timer hardware, per la generazione del PWM e la lettura dell'encoder;
- Inizializzazione delle interfacce di comunicazione (UART, I²C).

Una volta ultimata la configurazione grafica, l'IDE provvede alla generazione automatica del codice C di base, fornendo un'architettura software pre-inizializzata su cui lo sviluppatore può procedere direttamente all'implementazione della logica di controllo.

3.0.2 MATLAB

MATLAB, ambiente di calcolo numerico e linguaggio di programmazione sviluppato da MathWorks, è uno strumento ampiamente impiegato in ambito ingegneristico per l'analisi dei dati, lo sviluppo di algoritmi, la simulazione di sistemi di controllo e l'elaborazione dei segnali. All'interno di questo progetto, le funzionalità di plotting di MATLAB sono state impiegate specificamente per l'acquisizione e la visualizzazione dei dati in tempo reale, fornendo un supporto visivo per il monitoraggio delle variabili in gioco e la validazione del comportamento dinamico del veicolo.

3.0.3 Acquisizione dati via Terminale

Per la cattura e la verifica preliminare dei dati telemetrici inviati dal microcontrollore, ci si è avvalsi del terminale nativo di macOS. Sfruttando la comunicazione seriale (UART via USB) tra la scheda STM32 e il computer, l'interfaccia a riga di comando ha permesso di monitorare e registrare il flusso continuo di dati grezzi in uscita. Questo step ha consentito di verificare l'integrità, la frequenza e la corretta formattazione dei pacchetti dati, come i

conteggi dell'encoder o l'errore del PID, prima che questi venissero acquisiti ed elaborati dall'ambiente MATLAB per il plotting in tempo reale.

4 Implementazione del firmware

In questo capitolo verranno analizzati gli algoritmi di controllo sviluppati sul microcontrollore dal punto di vista della loro implementazione.

4.1 Diagramma di flusso

Si descrivere in maniera complessiva la logica del sistema attraverso il successivo diagramma:

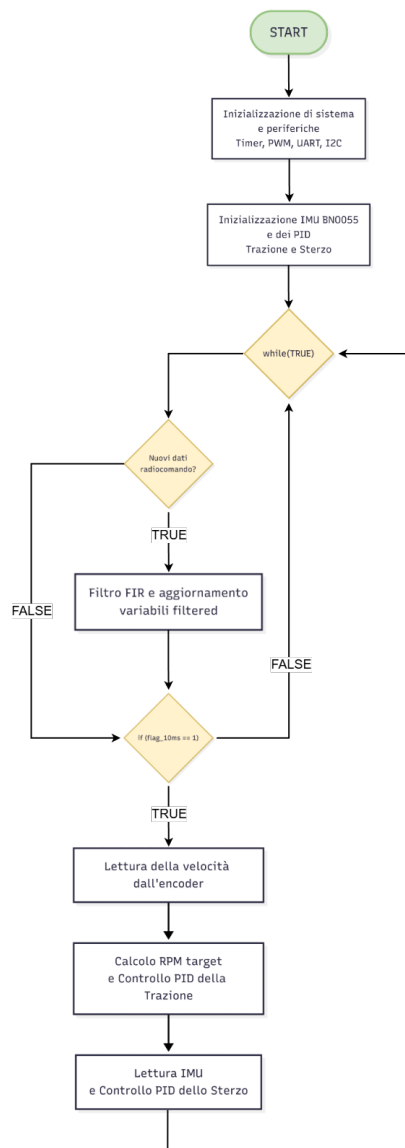


Figure 4.1: *Diagramma di flusso del codice sviluppato*

Si evidenzia come il flusso di esecuzione delle istruzioni del codice inizi con una fase preliminare di setup, in cui vengono inizializzate le periferiche di sistema e configurati i componenti essenziali, ovvero l'IMU e i due regolatori PID per il controllo della trazione e dello sterzo. Successivamente, il programma entra in un ciclo di esecuzione infinito, all'interno di questo ciclo, il flusso operativo dipende dall'esito di due blocchi condizionali principali. Il primo verifica costantemente la ricezione di nuovi comandi impartiti tramite il radiocomando: quando questa condizione è verificata, i segnali grezzi in ingresso vengono elaborati attraverso un filtro FIR (Finite Impulse Response), al fine di smorzarne le oscillazioni. Nel caso in cui non vi siano nuovi dati, l'algoritmo salta l'elaborazione del filtro. Il flusso prosegue poi verso la seconda condizione, la verifica della variabile temporale "flag 10ms": se tale variabile assume il valore 1, significa che è trascorso il periodo di campionamento prefissato di 10 millisecondi. In questa occorrenza, il sistema esegue in stretta sequenza le operazioni necessarie per il movimento del veicolo. Al termine di queste operazioni, o nel caso in cui il periodo di 10ms non sia ancora trascorso, il flusso ritorna all'inizio del ciclo per ripetere le verifiche.

4.2 Implementazione relativa al radiocomando

L'implementazione relativa al radiocomando rappresenta il fulcro di questa relazione: l'obiettivo del progetto è infatti proprio quello di gestire la comunicazione tra radiocomando e microcontrollore, al fine di gestire il moto del veicolo (trazione e sterzo).

4.2.1 Impostazioni iniziali timers e calcolo preliminare di frequenza e duty

In questa sottosezione si riportano le impostazioni iniziali dei Timers e il codice utilizzato per l'acquisizione dei primi valori di frequenza e duty fondamentali per le fasi di implementazione successive. Inizialmente si è analizzato il segnale PWM restituito da ogni canale del ricevitore. Per ottenere la frequenza ed il duty cycle tutti i canali, è stato utilizzato un interrupt in modalità InputCapture che consente di studiare un segnale preso come input, generato appunto dal ricevitore. Tramite la configurazione nel file ".ioc" di questo interrupt, è necessario specificare la "Polarity Section", ovvero a quale evento del segnale si è interessati: "Rising Edge" (fronte di salita), "Falling Edge" (fronte di discesa), "Both Edges" (fronte di salita e di discesa).

Impostazioni timer

I timer permettono di misurare il tempo e quindi di ottenere i valori desiderati dalla ricevente. Per l'utilizzo dell'Input Capture (sfruttando i canali PA0 e PA3 del TIM5) si è deciso di utilizzare un timer con un clock di 64 MHz, che viene diviso dal "Prescaler". Poiché a livello hardware il microcontrollore somma sempre 1 al valore del registro, per ottenere un clock effettivo di 1 MHz il parametro è stato settato a 63, calcolato mediante la formula $PSC = \frac{64.000.000}{1.000.000} - 1$.

Inoltre, poiché la frequenza minima leggibile è data da $\frac{TIMCLOCK}{COUNTERPERIOD}$, si è deciso di lasciare il "Counter Period" al massimo, ovvero 0xffffffff (corrispondente a 4294967295) essendo il TIM5 un timer a 32 bit.

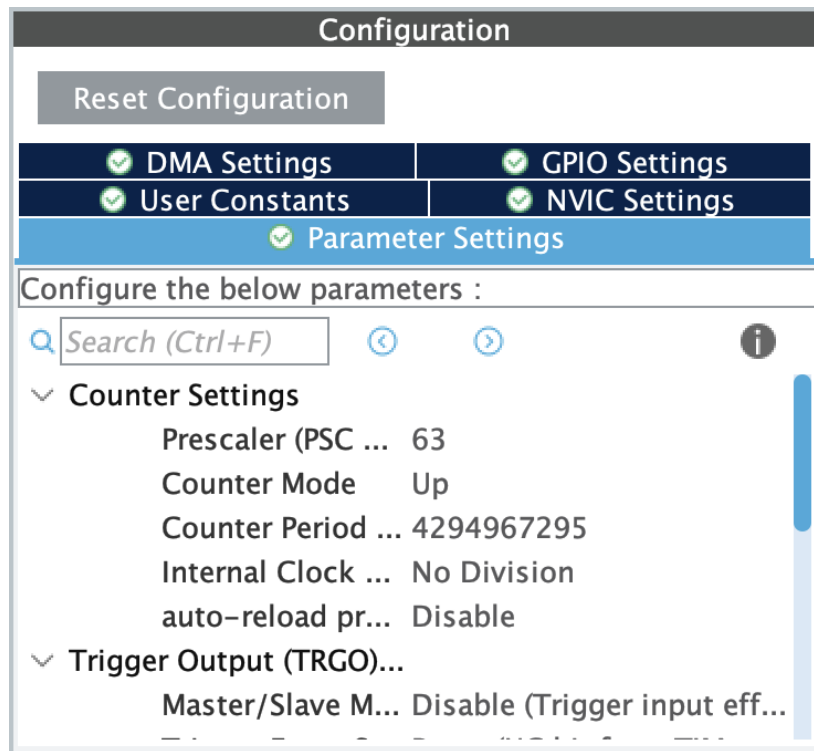


Figure 4.2: Impostazioni timer

Acquisizione del segnale PWM

Il segnale PWM in uscita dal ricevitore del radiocomando presenta una frequenza fissa (tipicamente 50 Hz, pari a un periodo di 20 ms). Pertanto, il calcolo della frequenza risulta irrilevante ai fini del controllo dinamico del veicolo. L'informazione vitale legata al comando impartito dal pilota risiede unicamente nella larghezza dell'impulso alto (T_{on}), che varia proporzionalmente al movimento delle levette in un intervallo tipicamente compreso tra 1000 μ s e 2000 μ s.

Per acquisire direttamente questo dato vitale, si è configurato il parametro "Polarity Selection" su "Both Edges" all'interno dell'ambiente di sviluppo, discostandosi da un approccio basato sul solo fronte di salita.

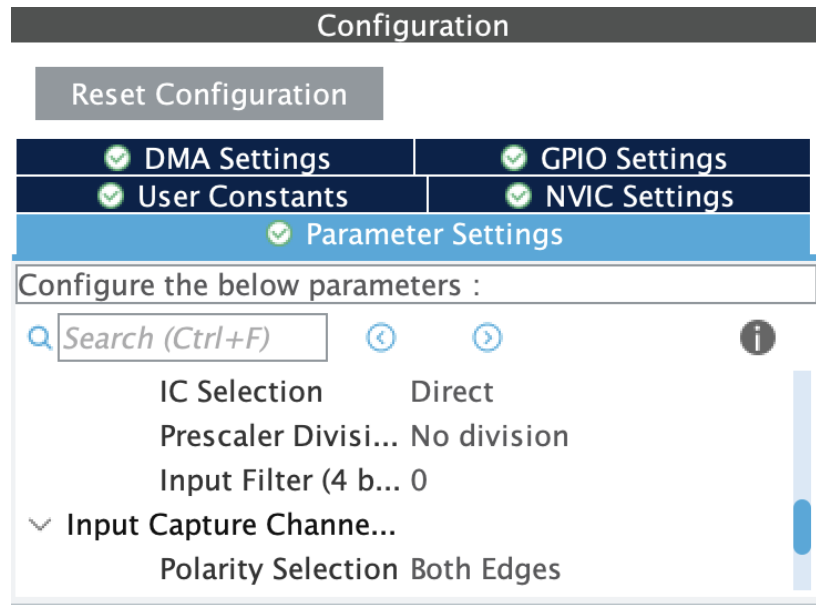


Figure 4.3: Configurazione Polarity Selection su Both Edges

La logica di acquisizione è gestita all'interno della funzione di callback `HAL_TIM_IC_CaptureCallback`. Tramite l'ausilio di flag di stato (come `Is_First_Captured_Steering`), il sistema salva il valore del contatore hardware al verificarsi del fronte di salita, commutando contestualmente la sensibilità del canale sul fronte opposto. Al successivo fronte di discesa, il contatore viene letto nuovamente per poterne calcolare la differenza temporale.

Poiché il timer opera con un clock effettivo di 1 MHz, la differenza algebrica tra le due letture restituisce la durata dell'impulso espressa direttamente in microsecondi. Tale valore viene salvato nelle variabili globali (`steering_us` e `throttle_us`) per essere impiegato come setpoint, a patto che superi il controllo di sicurezza implementato per scartare letture anomale esterne al range nominale (600–2400 μ s).

```

1 void HAL_TIM_IC_CaptureCallback(TIM_HandleTypeDef *htim)
2 {
3     // STERZO -> PA3 (TIM5 CH4)
4     if (htim->Instance == TIM5 && htim->Channel == HAL_TIM_ACTIVE_CHANNEL_4)
5     {
6         static uint32_t ic_val1_s = 0;
7         uint32_t ic_val2_s = 0;
8         uint32_t diff_steering = 0;
9
10        if (Is_First_Captured_Steering == 0)
11        {
12            ic_val1_s = HAL_TIM_ReadCapturedValue(htim, TIM_CHANNEL_4);
13            Is_First_Captured_Steering = 1;
14            __HAL_TIM_SET_CAPTUREPOLARITY(htim, TIM_CHANNEL_4,
15                                          TIM_INPUTCHANNELPOLARITY_FALLING);
16        }
17        else
18        {
19            ic_val2_s = HAL_TIM_ReadCapturedValue(htim, TIM_CHANNEL_4);

```

```

20     if (ic_val2_s > ic_val1_s)
21         diff_steering = ic_val2_s - ic_val1_s;
22     else
23         diff_steering = (0xFFFFFFFF - ic_val1_s) + ic_val2_s;
24
25     // Per lo sterzo FIR
26     if (diff_steering > 600 && diff_steering < 2400) {
27         steering_us = diff_steering;
28         new_rc_steering = 1;
29     }
30
31     Is_First_Captured_Steering = 0;
32     __HAL_TIM_SET_CAPTUREPOLARITY(htim, TIM_CHANNEL_4,
33         TIM_INPUTCHANNELPOLARITY_RISING);
34 }
35
36 // Trazione -> PA0 (TIM5 CH1)
37 if (htim->Instance == TIM5 && htim->Channel == HAL_TIM_ACTIVE_CHANNEL_1)
38 {
39     static uint32_t ic_val1_t = 0;
40     uint32_t ic_val2_t = 0;
41     uint32_t diff_throttle = 0;
42
43     if (Is_First_Captured_Throttle == 0)
44     {
45         ic_val1_t = HAL_TIM_ReadCapturedValue(htim, TIM_CHANNEL_1);
46         Is_First_Captured_Throttle = 1;
47         __HAL_TIM_SET_CAPTUREPOLARITY(htim, TIM_CHANNEL_1,
48             TIM_INPUTCHANNELPOLARITY_FALLING);
49     }
50     else
51     {
52         ic_val2_t = HAL_TIM_ReadCapturedValue(htim, TIM_CHANNEL_1);
53
54         if (ic_val2_t > ic_val1_t)
55             diff_throttle = ic_val2_t - ic_val1_t;
56         else
57             diff_throttle = (0xFFFFFFFF - ic_val1_t) + ic_val2_t;
58
59         if (diff_throttle > 600 && diff_throttle < 2400) {
60             throttle_us = diff_throttle;
61             new_rc_throttle = 1;
62         }
63
64         Is_First_Captured_Throttle = 0;
65         __HAL_TIM_SET_CAPTUREPOLARITY(htim, TIM_CHANNEL_1,
66             TIM_INPUTCHANNELPOLARITY_RISING);
67     }
68 }

```

Listing 4.1: "Codice per la lettura del segnale PWM tramite Input Capture"

Gestione degli Interrupt e ottimizzazione dei Timer

A differenza delle classiche implementazioni hardware che prevedono l'azzeramento forzato del contatore del timer ad ogni fronte di salita (metodologia che impedisce la lettura simultanea di più canali e costringe all'utilizzo di molteplici timer), in questo progetto si è optato per un approccio software non distruttivo.

Come si evince dalla configurazione di sistema in STM32CubeMX (Figura 4.4), il timer dedicato alla decodifica (TIM5) è attivamente impiegato sui canali 1 e 4 in modalità *Input Capture direct mode*, ma il suo contatore viene fatto avanzare in modo continuo (fino al suo limite massimo a 32 bit, pari a 4294967295) senza mai subire reset forzati. All'attivazione dell'interrupt, la *callback* si limita a "fotografare" il valore assoluto del registro (tramite la funzione `HAL_TIM_ReadCapturedValue`).

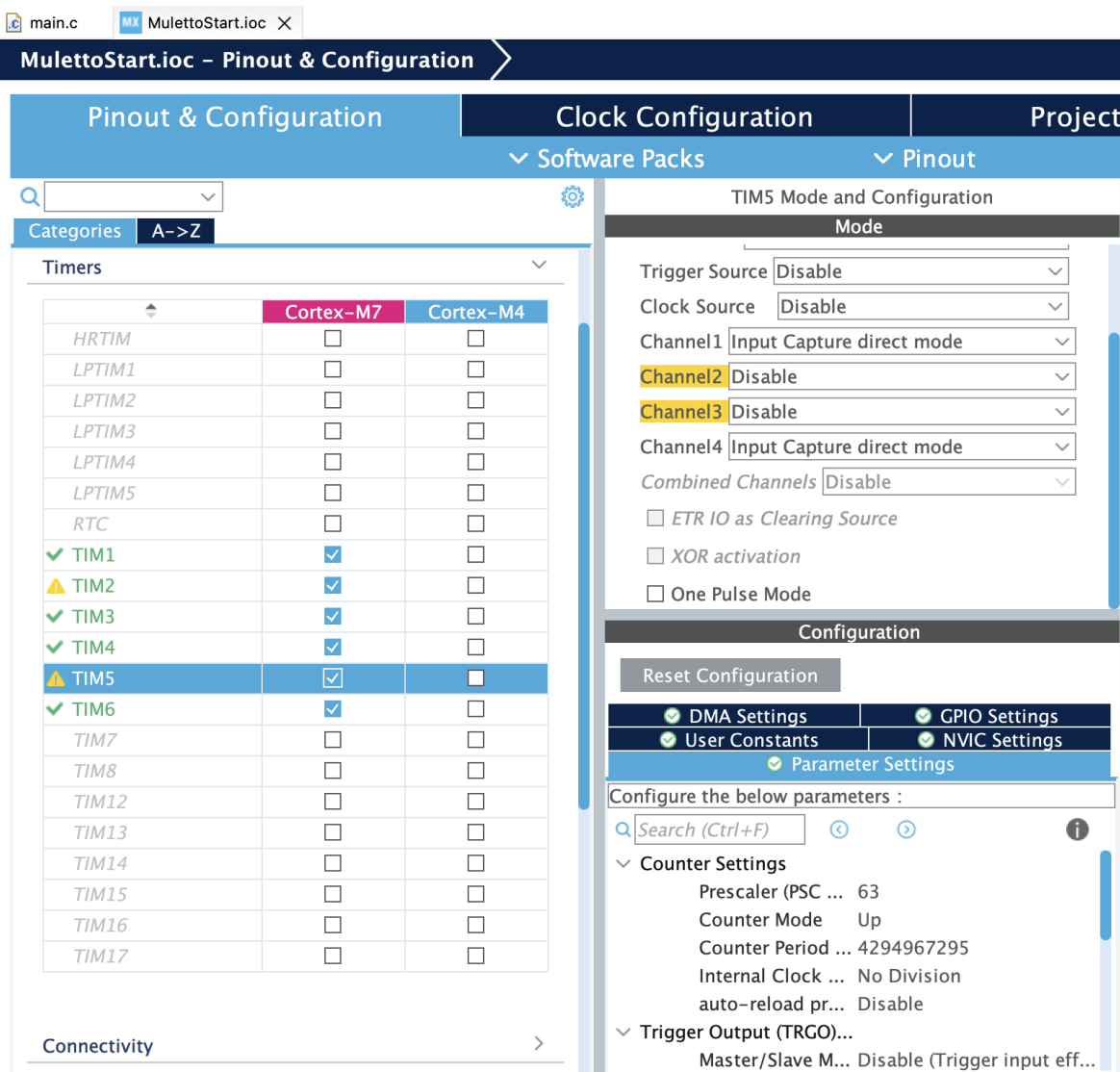


Figure 4.4: Configurazione del TIM5 con i canali 1 e 4 attivi

La durata dell'impulso viene così ricavata puramente per differenza matematica tra il timestamp di discesa e quello di salita. Per prevenire errori di calcolo nel raro caso in cui avvenga un overflow del timer proprio durante un impulso, l'algoritmo include un controllo condizionale che compensa il riavvio del conteggio sommando il complemento a due del limite massimo.

Questa architettura software garantisce numerosi vantaggi:

- **Ottimizzazione delle risorse:** consente di gestire in modo totalmente asincrono e indipendente l'acquisizione dei comandi Xa e Xb utilizzando un unico timer (TIM5) e due soli canali, riducendo il carico computazionale e liberando le restanti periferiche del microcontrollore.
- **Assenza di interferenze:** poiché il contatore principale non viene mai azzerato, l'arrivo di un interrupt sul canale dedicato a Xb non corrompe in alcun modo il conteggio temporale in corso sul canale di Xa, garantendo una precisione assoluta su entrambi i segnali in ingresso.
- **Flessibilità della polarità:** la commutazione dinamica della polarità, da Falling a Rising e viceversa, viene gestita direttamente all'interno della singola routine di interrupt.

Struttura codice principale

In questo progetto si è scelto di centralizzare l'acquisizione e il filtraggio dei comandi direttamente all'interno del file `main.c`. Questa architettura riduce l'overhead delle chiamate a funzioni, ottimizzando i tempi di esecuzione per il sistema di controllo in tempo reale. Il codice si articola in tre concetti fondamentali per il trattamento dei dati:

1. Struttura dati centralizzata

Invece di utilizzare strutture separate unicamente per la lettura temporale dei segnali, lo stato globale e le cinematiche della vettura sono state incapsulate in un'unica `struct` denominata `vehicleData`. Questa struttura funge da raccoglitore univoco per tutte le metriche fondamentali, semplificando enormemente la gestione della memoria e il passaggio di parametri alle funzioni di controllo.

```
1 typedef struct VehicleData {
2     // Trazione (Xa)
3     int counts;
4     int ref_count;
5     int delta_count;
6     float motor_speed_RPM;
7     float linear_speed_m_s;
8     float motor_speed_ref_RPM;
9     uint8_t motor_direction_ref;
10
11     // Sterzo (Xb)
12     double yaw_rate_rad_sec;
13     double yaw_rate_deg_sec;
14     double yaw_rate_ref_rad_sec;
15 } vehicleData;
```

Listing 4.2: "Definizione della struttura dati del veicolo"

2. Stabilizzazione e zona morta

I potenziometri fisici presenti sui radiocomandi commerciali sono intrinsecamente soggetti a lievi fluttuazioni elettriche e rumore meccanico. Se non filtrati, tali disturbi farebbero percepire al microcontrollore un comando variabile anche quando le levette sono a riposo (centrate idealmente a 1500 μ s), causando impercettibili movimenti dei motori. Per ovviare a questo problema, è stata implementata via software una "zona morta" di sicurezza. Come visibile nel codice, qualsiasi variazione del segnale di trazione (**Xa**) e di sterzo (**Xb**) che ricada nell'intorno del punto di neutro viene forzatamente ignorata, garantendo il perfetto arresto del veicolo e l'allineamento delle ruote anteriori.

```
1 // Zona morta per la trazione (Xa)
2 if(throttle_filtered > 1560.0f) {
3     target_dir = 1;
4     target_rpm = ((throttle_filtered - 1560.0f) * MAX_TARGET_RPM) /
5         433.0f;
6 }
7 else if(throttle_filtered < 1440.0f) {
8     target_dir = 0;
9     target_rpm = ((1440.0f - throttle_filtered) * MAX_TARGET_RPM) /
10         449.0f;
11 }
12 else {
13     target_rpm = 0.0f;
14     target_dir = 0;
15     pid_traction.lterm = 0;
16     pid_traction.e_old = 0;
17 }
18 // Zona morta per lo sterzo (Xb)
19 if (steering_filtered > 1460.0f && steering_filtered < 1560.0f) {
20     pid_steering.lterm = 0;
21     pid_steering.e_old = 0;
22     u_sterzo = 0.0f;
23 }
```

Listing 4.3: "Implementazione della zona morta per Xa e Xb"

3. Main Loop

Tutte queste operazioni avvengono costantemente all'interno del ciclo infinito `while(1)`. Qui i segnali grezzi intercettati dagli interrupt vengono passati prima attraverso un filtro FIR per regolarizzarne l'andamento, poi limitati dalle zone morte descritte in precedenza e infine mappati nei riferimenti (setpoint) da fornire agli anelli di controllo PID.

Operazioni di inizializzazione e avvio dei Timer

L'approccio implementato per l'avvio e la configurazione del sistema all'interno del `main.c` si basa su una logica del tutto asincrona e non bloccante. Le operazioni iniziali si limitano ad avviare le periferiche necessarie in sequenza, delegando l'intera logica di allineamento al fronte d'onda alla macchina a stati finiti (gestita tramite i flag software) implementata direttamente all'interno della routine di *interrupt*. In particolare, per quanto riguarda l'acquisizione dei segnali provenienti dal radiocomando (Xa per la trazione e Xb per lo sterzo), è sufficiente

abilitare gli *interrupt* legati all'Input Capture sui canali 1 e 4 del TIM5. Contestualmente, viene avviato il timer di base (TIM6) che scandisce il ciclo di controllo primario ogni 10 ms.

```
1 // Avvio del Timer per il ciclo di esecuzione a 10ms
2 HAL_TIM_Base_Start_IT(&htim6);
3 // Avvio degli interrupt di Input Capture per Xa (Trazione) e Xb (Sterzo)
4 HAL_TIM_IC_Start_IT(&htim5, TIM_CHANNEL_1);
5 HAL_TIM_IC_Start_IT(&htim5, TIM_CHANNEL_4);
6 // Inizializzazione delle strutture PID per trazione e sterzo
7 init_PID(&pid_traction, TRACTION_SAMPLING_TIME, MAX_U_TRACTION,
8         MIN_U_TRACTION);
9 tune_PID(&pid_traction, KP_TRACTION, KI_TRACTION, 0);
10 init_PID(&pid_steering, STEERING_SAMPLING_TIME, MAX_U_STEERING,
11         MIN_U_STEERING);
12 tune_PID(&pid_steering, KP_STEERING, KI_STEERING, 0);
```

Listing 4.4: "Avvio delle periferiche hardware e del sistema di controllo"

Questo paradigma architetturale snellisce il codice sorgente e riduce i tempi di accensione del sistema (*boot*), scongiurando inoltre il rischio di blocchi critici (*deadlock*) nel caso in cui il radiocomando sia spento al momento dell'accensione del microcontrollore. Essendo privo di attese bloccanti, il flusso principale continua la sua esecuzione naturale, inizializzando i sensori (come l'IMU via I²C) e posizionando le variabili di sicurezza nelle relative zone morte. Ciò garantisce che il veicolo venga mantenuto in uno stato stazionario sicuro fino all'effettiva ricezione del primo pacchetto dati valido.

Ciclo di esecuzione principale - Main loop

Una volta concluse le operazioni di inizializzazione e avviate le periferiche, il firmware entra nel ciclo di esecuzione infinito `while(1)`. La struttura di questo ciclo è stata progettata per separare logicamente l'acquisizione asincrona dei comandi radio dall'anello di controllo deterministico del veicolo. Il ciclo si divide in due macro-blocchi fondamentali:

1. **Acquisizione e validazione comandi:** non appena i flag `new_rc_throttle` e `new_rc_steering` segnalano la disponibilità di un nuovo dato temporale (Xa e Xb), il sistema provvede a validare e pre-filtrare i segnali grezzi, salvandoli nelle variabili globali di stato.
2. **Anello di controllo (10 ms):** sfruttando un flag hardware (`Flag_10ms`) generato dall'interrupt del TIM6, il sistema esegue il calcolo delle leggi di controllo a una frequenza fissa e predicibile (100 Hz). In questa fase vengono acquisiti i feedback dai sensori (Encoder e IMU), vengono generati i setpoint e calcolate le azioni di controllo PID.

Di seguito è riportata la struttura architetturale del ciclo principale:

```
1 while (1)
2 {
3     // --- 1. ACQUISIZIONE E VALIDAZIONE COMANDI RADIO ---
4     if(new_rc_throttle)
5     {
6         new_rc_throttle = 0;
7         // ... (Filtro e validazione segnale Xa) ...
8     }
```



```

9
10  if(new_rc_steering)
11  {
12      new_rc_steering = 0;
13      // ... (Filtro e validazione segnale Xb) ...
14  }
15
16  // --- 2. ANELLO DI CONTROLLO DETERMINISTICO (10 ms) ---
17  if (Flag_10ms == 1)
18  {
19      Flag_10ms = 0;
20
21      // Acquisizione feedback sensori (Encoder, BN0055)
22      // ...
23
24      // Mappatura comandi Xa e Xb in setpoint fisici (RPM e rad/s)
25      // ...
26
27      // Esecuzione PID Trazione e Sterzo
28      // ...
29
30      // Applicazione output ai motori e telemetria
31      // ...
32  }
33 }

```

Listing 4.5: "Struttura a macro-blocchi del loop infinito principale"

L'implementazione matematica dei filtri di validazione e le logiche di mappatura dei segnali Xa e Xb nei rispettivi setpoint fisici verranno analizzate in dettaglio nei successivi capitoli, dedicati al **Controllo della Trazione** e al **Controllo dello Sterzo**.

4.2.2 Controllo della trazione

In merito al controllo della trazione si è deciso di utilizzare un controllo a catena chiusa: si utilizzano come feedback le letture provenienti dall'encoder correggendo i risultati utilizzando un singolo controllore PID per regolare l'intensità della velocità, mentre il senso di marcia viene gestito separatamente tramite un apposito pin digitale. Il riferimento viene dato dal radiocomando tramite la levetta a destra. La velocità del veicolo aumenterà all'aumentare della "posizione" della levetta, così come la vettura andrà in retromarcia spingendo la leva verso il basso. All'interno del codice sono stati utilizzati diversi file di intestazione e file sorgente, per ottenere un codice modulare: al fine di svolgere i test in modo più preciso e in sicurezza, il limite massimo di potenza inviata al motore (Duty Cycle del segnale PWM) è stato bloccato al 20

File e strutture dati per il controllo

In questa sezione si riportano i file, le definizioni e le strutture utilizzate per il controllo della trazione e dello sterzo, commentandone le costanti e i prototipi di funzione.

DC_motor.h

Il modulo relativo al motore DC racchiude le definizioni e le funzioni necessarie per il pilo-

taggio dell'attuatore. Nello specifico, all'interno del codice vengono definiti e implementati i seguenti elementi:

```

1     #ifndef INC_DC_MOTOR_H_
2 #define INC_DC_MOTOR_H_
3
4 #include <PID.h>
5 #include <main.h>
6 #include "math.h"
7
8
9 #define V_MAX 7.5
10 #define V_MIN 1.0
11 #define GIRI_MIN_CONV 0.0006283185
12
13 float DegreeSec2RPM(float);
14
15 float Voltage2Duty(float);
16 uint8_t Ref2Direction(float);
17 void set_PWM_and_dir(uint32_t, uint8_t);
18 #endif /* INC_DC_MOTOR_H_ */

```

Listing 4.6: DC_motor.h

- **V_MAX**: definisce la tensione massima a cui il motore può lavorare, utilizzata come parametro di riferimento per il calcolo della percentuale di potenza da erogare.
- **DegreeSec2RPM**: è una funzione matematica adibita alla conversione cinematica dei valori fisici; trasforma la velocità angolare, calcolata in gradi al secondo, nella corrispondente velocità in RPM (giri al minuto).
- **Voltage2Duty**: rappresenta la funzione di conversione che trasforma lo sforzo di controllo, in Volt, nella rispettiva percentuale di Duty Cycle del segnale PWM.
- **set_PWM_and_dir**: è la funzione operativa che si occupa di inviare fisicamente il segnale PWM calcolato al timer hardware del microcontrollore e di commutare lo stato del pin digitale responsabile per la direzione di marcia.

Parametri dell'encoder

Per quanto riguarda l'acquisizione della velocità e dello spazio percorso, il sistema fa affidamento sui parametri fisici dell'encoder. A livello di architettura del software, si è scelto di non mantenere un file di intestazione isolato per questo sensore, le sue costanti fondamentali sono state invece accorpate all'interno del file `DC_motor.h`.

```

1 //ENCODER
2 #define ENCODER_PPR 2048
3 #define GEARBOX_RATIO 1
4 #define ENCODER_COUNTING_MODE 4

```

Listing 4.7: DC_motor.h

- **ENCODER_PPR** (Pulses Per Revolution): indica la risoluzione nominale del sensore, il quale genera 2048 impulsi per ogni giro completo del proprio albero.

- **GEARBOX_RATIO**: rappresenta il rapporto di riduzione della trasmissione meccanica. Essendo il valore impostato a 1, si assume che l'albero dell'encoder e l'albero motore ruotino alla medesima velocità.
- **ENCODER_COUNTING_MODE**: specifica la modalità di lettura configurata sul Timer hardware. Il valore 4 indica una lettura in quadratura su tutti i fronti (salita e discesa) per entrambi i canali (A e B), garantendo la massima risoluzione possibile nella stima del delta angolare.

Struttura dati dello stato del veicolo

La struttura dati utilizzata per la memorizzazione dello stato della vettura è dichiarata e gestita integralmente all'interno del file `main.c`. Al suo interno vengono raggruppate tutte le variabili di stato e le letture provenienti dai sensori di bordo necessarie al ciclo di controllo. La struttura è divisa logicamente in due blocchi operativi principali:

```

1 typedef struct VehicleData {
2     //Trazione
3     int counts;
4     int ref_count;
5     int delta_count;
6     float delta_angle_deg; // [deg]
7     float motor_speed_deg_sec; // [deg/s]
8     float motor_speed_RPM; // RPM
9     float linear_speed_m_s; // [m/s]
10    float motor_speed_ref_RPM; // RPM
11    uint8_t motor_direction_ref; //Sterzo
12    double yaw_rate_rad_sec; // [rad/s]
13    double yaw_rate_deg_sec; // [deg/s]
14    double yaw_rate_ref_rad_sec; // [rad/s]
15 } vehicleData;
```

Listing 4.8: Main.c

Parametri di trazione

- **counts**: memorizza i conteggi grezzi correnti letti dal registro del Timer hardware collegato all'encoder (TIM4 → CNT).
- **ref_count**: rappresenta un valore di conteggio di riferimento fisso (impostato a metà del periodo del timer) utilizzato ad ogni ciclo per evitare problemi di overflow o underflow durante la lettura.
- **delta_count**: indica la variazione netta dei conteggi (gli impulsi dell'encoder) avvenuta nell'ultimo ciclo di campionamento rispetto al riferimento.
- **delta_angle_deg**: rappresenta lo spostamento angolare del rotore, calcolato e convertito in gradi, avvenuto nell'intervallo di campionamento.
- **motor_speed_deg_sec**: è la velocità angolare attuale del motore, calcolata cinematicamente ed espressa in gradi al secondo.

- `motor_speed_RPM`: è la velocità di rotazione del motore convertita in RPM (Giri al Minuto); questa è la variabile di feedback (realtà) mandata in pasto al controllore PID della trazione.
- `linear_speed_m_s`: memorizza la velocità lineare effettiva del veicolo su strada, espressa direttamente in m/s (metri al secondo).
- `motor_speed_ref_RPM`: indica il setpoint, ovvero la velocità di riferimento in RPM richiesta dal pilota, derivata dalla mappatura proporzionale del segnale del radiocomando.
- `motor_direction_ref`: è una variabile intera (0 o 1) che memorizza il verso di rotazione richiesto per l'attuatore, definendo quindi se il veicolo deve procedere in marcia avanti o in retromarcia.

Parametri di sterzo e cinematica

- `yaw_rate_rad_sec`: memorizza il tasso di imbardata reale (la velocità di rotazione del veicolo attorno al proprio asse verticale z) acquisito tramite il filtro FIR dal giroscopio dell'unità IMU (BNO055), espresso in radianti al secondo.
- `yaw_rate_deg_sec`: rappresenta il medesimo dato di imbardata del veicolo, ma calcolato ed espresso in gradi al secondo.
- `yaw_rate_ref_rad_sec`: memorizza la velocità di imbardata di riferimento, che corrisponde all'obiettivo di rotazione direzionale richiesto dal segnale del radiocomando.

PID.h

Nel file `PID.h` viene definita la struttura dati astratta del controllore PID, essenziale per la logica di calcolo del sistema. A livello architetturale, vengono istanziati due controllori PID distinti: uno, `pid_traction`, dedicato alla regolazione della velocità in entrambi i sensi di marcia e l'altro, `pid_steering`, per il controllo direzionale dello sterzo. Nel file sorgente `PID.c` sono poi implementate le funzioni per l'inizializzazione (`init_PID`), la taratura (`tune_PID`) e il calcolo iterativo della legge di controllo (`PID_controller`).

```

1 typedef struct PID{
2     float Kp; //guadagno proporzionale
3     float Ki; //guadagno integrale
4     float Kd; //guadagno derivativo
5
6     float Tc; //periodo
7     float u_max; //limite superiore
8     float u_min; //limite inferiore
9
10    float e_old;
11    float Iterm;
12 }PID;
13
14 void init_PID(PID*, float, float, float);
15 void tune_PID(PID*,float,float,float);
16 float PID_controller(PID*,float,float);

```

Listing 4.9: PID.h

Di seguito si analizzano nel dettaglio i parametri operativi della struttura:

- **Tc - Periodo di campionamento:** rappresenta l'intervallo di tempo esatto tra due esecuzioni consecutive dell'algoritmo.
- **u_max e u_min - limiti di saturazione:** definiscono il tetto massimo e il limite minimo consentiti per l'uscita del controllore. Qualora il calcolo matematico superi queste soglie, la funzione di controllo interviene tagliando l'uscita per mantenerla entro i limiti di sicurezza.
- **e_old - errore precedente:** memorizza il valore dell'errore misurato nel ciclo esecutivo appena trascorso. È necessario per calcolare l'azione derivativa, la quale valuta proprio la differenza tra l'errore nuovo e quello vecchio.
- **Iterm - termine integrale:** funge da registro di memoria che somma e conserva lo storico degli errori nel tempo. Per ragioni di stabilità, la funzione `PID_controller` integra una logica di *clamping anti-windup*: se il controllore raggiunge i limiti di saturazione, l'aggiornamento di questa variabile viene bloccato per impedire uno sproporzionato accumulo dell'errore.

PID.c

Il file sorgente `PID.c` contiene l'implementazione logica e matematica delle funzioni necessarie al funzionamento del controllore a catena chiusa. Questo modulo gestisce l'intero ciclo di vita del PID, dall'impostazione iniziale fino al calcolo iterativo dello sforzo di controllo.

```
1 #include "PID.h"
2 #include <stdio.h>
3
4 void init_PID(PID* p, float Tc, float u_max, float u_min){
5     p->Tc = Tc;
6     p->u_max = u_max;
7     p->u_min = u_min;
8     p->Iterm = 0;
9     p->e_old = 0;
10 }
11
12 void tune_PID(PID*p, float Kp, float Ki, float Kd){
13     p->Kp = Kp;
14     p->Ki = Ki;
15     p->Kd = Kd;
16 }
17
18 float PID_controller(PID* p, float y, float r){
19     float u;
20     float newIterm;
21     float e = 0;
22
23     e = r-y;
24
25
26     float Pterm = p->Kp*e;
27     newIterm = p->Iterm + (p->Ki)*p->Tc*p->e_old;
```

```

28     float Dterm = (p->Kd/p->Tc)*(e - p->e_old);
29
30     p->e_old = e;
31
32     u = Pterm + newIterm + Dterm;
33
34     if(u > p->u_max){
35         u = p->u_max;
36     } else if(u < p->u_min){
37         u = p->u_min;
38     } else {
39         p->Iterm = newIterm;
40     }
41
42     return u;
43 }

```

Listing 4.10: PID.c

4.3 Implementazione del Controllo dello Sterzo

In questa sezione si analizza l’implementazione del controllo dello sterzo, fondamentale per gestire la dinamica laterale e direzionale del veicolo. L’architettura di questo modulo si basa su un sistema di controllo retroazionato in catena chiusa per garantire un’elevata precisione di manovra. Il focus è posto sull’ottenere una proporzionalità tra la posizione della leva del radiocomando (X_b) e il tasso di imbardata (*yaw rate*) del veicolo, ovvero la sua velocità di rotazione sull’asse verticale. In particolare, si impone una traiettoria rettilinea (tasso di rotazione nullo) in corrispondenza della leva in posizione neutra, mentre spostando la leva a destra o a sinistra viene calcolata una velocità di imbardata di riferimento (*yaw_rate_ref*) proporzionale allo spostamento. Il controllo della sterzata avviene tramite l’ausilio di un controllore PID dedicato (*pid_steering*). Il sistema acquisisce in tempo reale la velocità angolare effettiva del veicolo (*feedback*) leggendo i dati del giroscopio forniti dall’unità inerziale IMU BNO055. Il controllore PID confronta il riferimento richiesto dal pilota con il dato reale dell’IMU e calcola lo sforzo di controllo (*u_sterzo*) necessario per annullare l’errore direzionale. Infine, la variabile di uscita calcolata dal PID viene tradotta nel segnale di comando per il servomotore. La posizione angolare dell’albero dell’attuatore è determinata dalla larghezza dell’impulso del segnale PWM fornito al componente, il cui *duty cycle* ruota attorno a un valore centrale di calibrazione che corrisponde alla marcia in rettilineo. L’algoritmo si occupa di aggiornare continuamente questo segnale in base all’uscita del controllore, assicurandosi al contempo che l’angolo calcolato non superi mai i limiti meccanici di sicurezza, bloccando la saturazione del servomotore all’interno del *range* consentito tra -20 e +20 gradi.

Configurazione del Timer per l’attuazione dello sterzo (PWM)

Per controllare fisicamente la posizione angolare del servomotore dello sterzo, è necessario generare un segnale PWM con caratteristiche ben precise. Per questo compito è stato scelto il TIM1, assegnato al core *Cortex-M7*, configurando il Canale 2 in modalità “PWM Generation CH2”. I parametri fondamentali del timer sono stati calcolati per ottenere un segnale con

una frequenza di 50 Hz (corrispondente a un periodo di 20 ms), che rappresenta lo standard industriale per il pilotaggio dei servomotori. Analizzando la configurazione:

- **Prescaler (PSC):** è stato impostato al valore 63. Ipotizzando un *clock* di base del timer a 64 MHz, questo divisore ($63 + 1$) abbassa la frequenza di conteggio esattamente a 1 MHz. Questa scelta fa sì che ogni singolo incremento del timer corrisponda esattamente a 1 microsecondo (μs), facilitando enormemente la gestione matematica delle ampiezze degli impulsi nel codice.
- **Counter Period:** è stato impostato a 19999. Il timer è configurato in modalità “Up”, quindi conta da 0 a 19999, compiendo 20000 passi prima di azzerarsi. Considerando la risoluzione temporale di 1 microsecondo ricavata dal Prescaler, il periodo totale del segnale PWM risulta essere esattamente di 20000 microsecondi, ovvero 20 millisecondi (50 Hz).

Per modificare dinamicamente il *duty cycle* (e di conseguenza controllare l'angolo di sterzata del veicolo), il firmware agisce direttamente sul registro di comparazione associato al canale utilizzato, ovvero TIM1-CCR2. Grazie a questa configurazione temporale così pulita, la scrittura nel registro è diretta: per posizionare il servomotore è sufficiente assegnare alla variabile del registro il valore esatto in microsecondi calcolato dall'algoritmo di controllo (tipicamente compreso nel *range* operativo 1000 μs - 2000 μs), garantendo una risposta istantanea e precisa della meccanica.

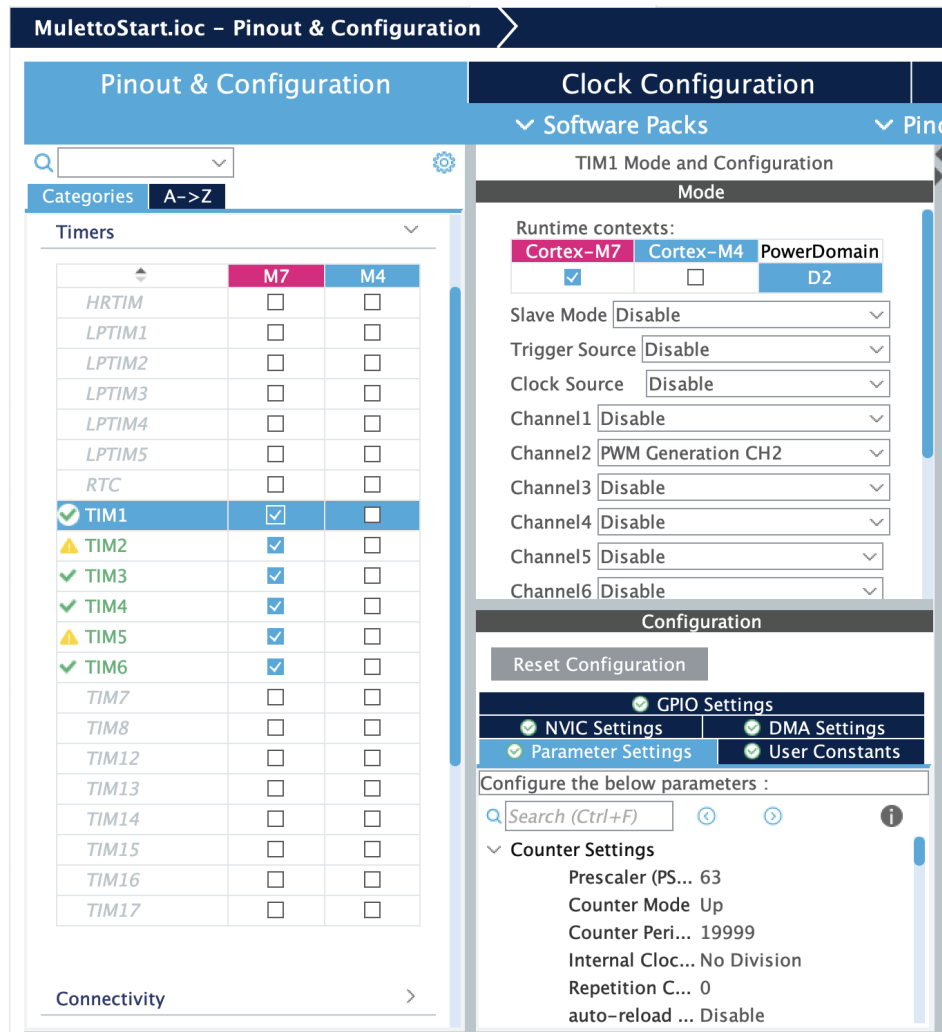


Figure 4.5: Configurazione del TIM1 per l'attuazione dello sterzo

servo_motor.h

Questo file di intestazione contiene il prototipo della funzione che si occuperà di generare il segnale PWM da fornire al servomotore, più altre costanti utili per la gestione dell'angolo di sterzata.

```

1 #include <main.h>
2
3 #ifndef INC_SERVO_MOTOR_H_
4 #define INC_SERVO_MOTOR_H_
5
6 #define DRITTO 92
7 #define MAX_ANGOLO 20
8 #define MIN_ANGOLO -20
9 #define PI 3.141592654
10
11 void servo_motor(float);

```



```

12
13 #endif /* INC_SERVO_MOTOR_H_ */

```

Listing 4.11: servo_motor.h

I valori costanti DRITTO, MAX_ANGOLO e MIN_ANGOLO, che dipendono dal tipo di servomotore utilizzato e dalla meccanica del veicolo, sono stati ottenuti sperimentalmente e rappresentano gli angoli di massima sterzata e quello neutro. Nello specifico, il valore di riposo impostato a 92 (anziché al teorico 90) è frutto di una calibrazione meccanica sul campo, necessaria per compensare i fisiologici giochi della tiranteria e garantire un moto della vettura perfettamente rettilineo quando il comando è azzerato.

bn055_STM.h

Questo file di intestazione funge da livello di astrazione (*wrapper*) tra il driver di base del sensore inerziale BNO055 e le librerie hardware HAL del microcontrollore STM32. Al suo interno sono implementate le funzioni fondamentali per gestire la comunicazione fisica bidirezionale tramite il bus I2C e per adattare le tempistiche di ritardo all'ambiente di esecuzione.

```

1 #ifndef INC_BNO055_STM32_H_
2 #define INC_BNO055_STM32_H_
3
4 #ifdef __cplusplus
5     extern "C" {
6 #endif
7
8 // Includiamo main.h per importare in automatico stm32h7xx_hal.h
9 #include "main.h"
10
11 #ifdef FREERTOS_ENABLED
12 #include "FreeRTOS.h"
13 #include "task.h"
14 #include "cmsis_os.h"
15 #endif
16
17 #include "bno055.h"
18 #include <stdio.h>
19
20 // Puntatore all'handle dell'I2C (sara inizializzato con &hi2c1)
21 I2C_HandleTypeDef *_bno055_i2c_port;
22
23 void bno055_assignI2C(I2C_HandleTypeDef *hi2c_device) {
24     _bno055_i2c_port = hi2c_device;
25 }
26 void bno055_delay(int time) {
27 #ifdef FREERTOS_ENABLED
28     osDelay(time);
29 #else
30     HAL_Delay(time);
31 #endif
32 }
33
34 void bno055_writeData(uint8_t reg, uint8_t data) {
35     uint8_t txdata[2] = {reg, data};
36     uint8_t status;

```

```

37
38 // Scrittura via I2C
39 status = HAL_I2C_Master_Transmit(_bno055_i2c_port, BNO055_I2C_ADDR << 1,
40                                     txdata, sizeof(txdata), 10);
41 if (status == HAL_OK) {
42     return;
43 }
44
45 // Gestione Status
46 if (status == HAL_ERROR) {
47     printf("HAL_I2C_Master_Transmit HAL_ERROR\r\n");
48 } else if (status == HAL_TIMEOUT) {
49     printf("HAL_I2C_Master_Transmit HAL_TIMEOUT\r\n");
50 } else if (status == HAL_BUSY) {
51     printf("HAL_I2C_Master_Transmit HAL_BUSY\r\n");
52 } else {
53     printf("Unknown status data %d\r\n", status);
54 }
55
56 // Gestione Errori specifici
57 uint32_t error = HAL_I2C_GetError(_bno055_i2c_port);
58 if (error == HAL_I2C_ERROR_NONE) {
59     return;
60 } else if (error == HAL_I2C_ERROR_BERR) {
61     printf("HAL_I2C_ERROR_BERR\r\n");
62 } else if (error == HAL_I2C_ERROR_ARLO) {
63     printf("HAL_I2C_ERROR_ARLO\r\n");
64 } else if (error == HAL_I2C_ERROR_AF) {
65     printf("HAL_I2C_ERROR_AF\r\n");
66 } else if (error == HAL_I2C_ERROR_OVR) {
67     printf("HAL_I2C_ERROR_OVR\r\n");
68 } else if (error == HAL_I2C_ERROR_DMA) {
69     printf("HAL_I2C_ERROR_DMA\r\n");
70 } else if (error == HAL_I2C_ERROR_TIMEOUT) {
71     printf("HAL_I2C_ERROR_TIMEOUT\r\n");
72 }
73
74 // Gestione Macchina a Stati (Pulita per STM32H7)
75 HAL_I2C_StateTypeDef state = HAL_I2C_GetState(_bno055_i2c_port);
76 if (state == HAL_I2C_STATE_RESET) {
77     printf("HAL_I2C_STATE_RESET\r\n");
78 } else if (state == HAL_I2C_STATE_READY) {
79     printf("HAL_I2C_STATE_READY\r\n");
80 } else if (state == HAL_I2C_STATE_BUSY) {
81     printf("HAL_I2C_STATE_BUSY\r\n");
82 } else if (state == HAL_I2C_STATE_BUSY_TX) {
83     printf("HAL_I2C_STATE_BUSY_TX\r\n");
84 } else if (state == HAL_I2C_STATE_BUSY_RX) {
85     printf("HAL_I2C_STATE_BUSY_RX\r\n");
86 } else if (state == HAL_I2C_STATE_LISTEN) {
87     printf("HAL_I2C_STATE_LISTEN\r\n");
88 } else if (state == HAL_I2C_STATE_BUSY_TX_LISTEN) {
89     printf("HAL_I2C_STATE_BUSY_TX_LISTEN\r\n");
90 } else if (state == HAL_I2C_STATE_BUSY_RX_LISTEN) {
91     printf("HAL_I2C_STATE_BUSY_RX_LISTEN\r\n");
92 } else if (state == HAL_I2C_STATE_ABORT) {

```

```

93     printf("HAL_I2C_STATE_ABORT\r\n");
94 }
95 }
96
97 void bno055_readData(uint8_t reg, uint8_t *data, uint8_t len) {
98     HAL_StatusTypeDef ret;
99     ret = HAL_I2C_Master_Transmit(_bno055_i2c_port, BN0055_I2C_ADDR << 1, &
    reg, 1, 100);
100    ret = HAL_I2C_Master_Receive(_bno055_i2c_port, BN0055_I2C_ADDR << 1, data
    , len, 100);
101
102    if (ret == HAL_OK) {
103        return;
104    }
105
106    if (ret == HAL_ERROR) {
107        printf("HAL_I2C_Master_Transmit/Receive HAL_ERROR\r\n");
108    } else if (ret == HAL_TIMEOUT) {
109        printf("HAL_I2C_Master_Transmit/Receive HAL_TIMEOUT\r\n");
110    } else if (ret == HAL_BUSY) {
111        printf("HAL_I2C_Master_Transmit/Receive HAL_BUSY\r\n");
112    } else {
113        printf("Unknown status data %d\r\n", ret);
114    }
115 }
116
117 #ifdef __cplusplus
118 }
119 #endif
120
121 #endif /* INC_BN0055_STM32_H_ */

```

Listing 4.12: bno055_STM.h

Il codice è strutturato per offrire la massima flessibilità e solidità architetturale. L’inserimento delle direttive per il C/C++ garantisce un’elevata portabilità del modulo, mentre la gestione condizionale di FreeRTOS fa in modo che la funzione di ritardo adotti automaticamente istruzioni non bloccanti (come `osDelay`) qualora sia attivo un sistema operativo in tempo reale. Inoltre, la comunicazione I2C è totalmente indipendente dallo specifico hardware utilizzato: ciò è reso possibile dall’impiego del puntatore globale `_bno055_i2c_port`, che viene configurato dinamicamente in fase di avvio del sistema. Per concludere, le funzioni dedicate alla lettura e alla scrittura sono dotate di un meccanismo di diagnostica estremamente preciso. In caso di anomalie di comunicazione sul bus, il firmware analizza i registri del microcontrollore e trasmette sul terminale seriale la natura esatta dell’errore hardware (come ad esempio *timeout*, collisioni o blocchi della macchina a stati), semplificando in modo decisivo le procedure di *debug*.

4.3.1 servo_motor.c

Il file sorgente `servo_motor.c` contiene l’implementazione pratica della funzione responsabile dell’attuazione fisica dello sterzo. L’intera logica di questa funzione è stata ingegnerizzata mettendo al primo posto la sicurezza meccanica e l’efficienza di calcolo. Inizialmente, la funzione riceve in ingresso l’angolo di sterzata desiderato, calcolato dall’algoritmo di controllo

a partire dal riferimento Xb. Prima di qualsiasi elaborazione, questo valore viene vincolato rigidamente entro i limiti fisici della vettura (MIN_ANGOLO e MAX_ANGOLO). Questo *clamping* previene sforzi eccessivi e rotture dei tiranti meccanici nel caso di *setpoint* anomali. Successivamente, l'angolo relativo appena verificato viene sommato alla costante DRITTO, che rappresenta l'offset meccanico del centro. In questo modo il sistema passa da un riferimento relativo a un angolo reale assoluto, che il braccetto del servomotore deve fisicamente raggiungere. Questo angolo assoluto viene poi convertito nel tempo di accensione dell'impulso PWM (t_on) tramite un'equazione lineare. I coefficienti numerici presenti nella formula (come 1815.0, 57.0 e 7.3255814) non sono arbitrari, ma derivano da una rigorosa calibrazione sperimentale effettuata direttamente sulla risposta del vostro specifico servomotore, garantendo una precisione di posizionamento elevatissima. Come ulteriore livello di protezione, il tempo calcolato in microsecondi subisce una saturazione hardware, venendo limitato tra i 500 microsecondi e i 2500 microsecondi. Questi valori rappresentano gli estremi operativi standard per i servomotori da modellismo; inviare segnali esterni a questo intervallo causerebbe il surriscaldamento del motore interno e la conseguente rottura dell'attuatore. Infine, sfruttando la configurazione a 1 MHz del timer (dove un singolo incremento equivale esattamente a un microsecondo), il valore finale calcolato viene tradotto in un numero intero a 32 bit e sovrascritto direttamente nel registro hardware CCR2 del TIM1. Questo azionamento diretto a basso livello permette di aggiornare il *duty cycle* istantaneamente, annullando di fatto le latenze software.

```

1 #include "servo_motor.h"
2
3 void servo_motor(float deg_angle)
4 {
5     //Limiti sicurezza
6     if(deg_angle < MIN_ANGOLO)
7         deg_angle = MIN_ANGOLO;
8     else if (deg_angle > MAX_ANGOLO)
9         deg_angle = MAX_ANGOLO;
10
11     //Calcolo angolo reale
12     float conv_angolo = deg_angle + DRITTO;
13
14     //Calcolo in microsecondi
15     float t_on = 1815.0 - (conv_angolo - 57.0) * 7.3255814;
16
17     //Controllo di sicurezza(500us - 2500us)
18     if (t_on < 500.0) {
19         t_on = 500.0;
20     } else if (t_on > 2500.0) {
21         t_on = 2500.0;
22     }
23
24     //Invio diretto al timer
25     TIM1->CCR2 = (uint32_t)t_on;
26 }

```

Listing 4.13: servo_motor.c

Calibrazione meccanica dello sterzo

Prima di eseguire i test dinamici del veicolo, è stato necessario effettuare una calibrazione preliminare del servomotore. Questa procedura operativa è indispensabile per determinare empiricamente le costanti `DRITTO`, `MAX_ANGOLO` e `MIN_ANGOLO` definite all'interno del file di intestazione `servo_motor.h`. Questi valori devono essere ricavati ogni volta che il servomotore viene montato sulla vettura, in quanto non è nota la posizione iniziale del servo prima di diventare solidale al sistema di sterzo della macchina. Il primo passo della calibrazione consiste nel trovare l'angolo neutrale. Per farlo, si interviene sul valore della costante `DRITTO` (fissata nel nostro caso a 92) effettuando delle prove finché le ruote anteriori non assumono una posizione perfettamente rettilinea. Trovare l'angolo massimo e minimo, invece, risulta più difficoltoso poiché i movimenti del servomotore non sono completamente fluidi e osservabili per ogni grado di rotazione in quanto affetti da un attrito statico che deve essere superato. Per aggirare questo problema, si è deciso di sviluppare una routine di test temporanea che incrementa l'angolo del servomotore in modo continuato, aumentandolo di un grado ogni secondo. Questo espediente permette di osservare la sterzata in tempo reale e stabilire visivamente il limite oltre il quale le ruote non riescono più a muoversi per via del finecorsa meccanico. Durante questa procedura, è molto importante tenere traccia in tempo reale dell'angolo del servomotore, stampandolo di volta in volta sul terminale. Mantenere uno storico di questi valori è fondamentale perché, attraverso prove ripetute, sarà possibile definire l'angolo massimo reale facendo la media tra tutti quelli trovati e ignorando in questo modo eventuali errori dovuti alle condizioni non ideali in cui si lavora (presenza di attrito e resistenze parassite). A livello di codice, ci si è discostati dall'approccio del gruppo precedente (che gestiva l'incremento temporale all'interno della routine di *interrupt*). Per mantenere il sistema leggero e coerente con la nostra architettura, la temporizzazione del test sfrutta il `Flag_10ms` già presente nel `while(1)`. Utilizzando un contatore ausiliario, si attende il passaggio di 100 cicli (corrispondenti a 1 secondo esatto) per aggiornare l'angolo. Ecco il frammento di codice utilizzato per il test di calibrazione nel ciclo principale:

```
1  /* Variabili di test definite fuori dal while(1) */
2  int contatore_secondo = 0;
3  float angolo_test = 0.0f;
4
5  /* All'interno del ciclo while(1) */
6  if (Flag_10ms == 1)
7  {
8      // ... eventuale altro codice a 10ms ...
9
10     contatore_secondo++;
11
12     // Scatta ogni 100 * 10ms = 1000ms (1 secondo)
13     if (contatore_secondo >= 100)
14     {
15         contatore_secondo = 0;
16         angolo_test++;
17
18         servo_motor(angolo_test);
19         printf("Angolo attuale: %f\r\n", angolo_test);
20     }
```

21 }

Listing 4.14: Loop del test per la calibrazione del servomotore

Per testare l'angolo minimo di sterzo (quindi dalla parte opposta) sarà sufficiente sostituire `angolo_test++`; con `angolo_test--`;

5 Test e risultati

La presente sezione è dedicata all'analisi sperimentale e alla validazione sul campo degli algoritmi di controllo implementati sul microcontrollore. Al fine di valutare in modo completo e rigoroso le performance dinamiche del prototipo, si avranno due macro-aree fondamentali. Dapprima verranno esposti e discussi i risultati relativi al controllo della trazione e, successivamente, l'attenzione si sposterà sulla validazione del controllo direzionale dello sterzo, esaminando la risposta del sistema ai riferimenti di imbardata, i limiti fisici della cinematica del veicolo e l'impatto del filtraggio digitale sull'affidabilità della telemetria.

5.1 Trazione

Il controllo della velocità longitudinale è stato affidato a un regolatore di tipo PI (Proporzionale-Integrale), i cui parametri sono stati definiti empiricamente e affinati sul campo per garantire una risposta rapida e priva di errore a regime. Le costanti di guadagno ottimali, il tempo di campionamento dell'anello di controllo e i limiti di saturazione simmetrica sono stati definiti all'interno del firmware come segue:

```
1 //TRACTION PID
2 #define TRACTION_SAMPLING_TIME 0.01 //[s]
3 #define MAX_U_TRACTION 2.6 // [V]
4 #define MIN_U_TRACTION -2.6 // [V]
5 #define KP_TRACTION 0.006
6 #define KI_TRACTION 0.002
```

Listing 5.1: "Configuration.h"

Per l'analisi delle performance del sistema e l'estrapolazione dei risultati visivi, i dati sono stati acquisiti via seriale utilizzando anche il terminale per monitorare e validare preliminarmente il corretto flusso delle stringhe di testo in uscita dal microcontrollore. L'acquisizione definitiva e il tracciamento in tempo reale sono stati eseguiti su MATLAB con il seguente script:

```
1 % 1. Configurazione Porta Seriale
2 port_name = '/dev/tty.usbmodem1203';
3 baud_rate = 115200;
4
5 if exist('s', 'var')
6     clear s;
7 end
8
9 s = serialport(port_name, baud_rate);
10 s.Timeout = 10;
11 configureTerminator(s, "CR/LF");
12 flush(s);
13
14 % 2. Configurazione del Grafico
15 figure('Name', 'Telemetria Trazione', 'NumberTitle', 'off');
```

```

16 h_ref = animatedline('Color', 'r', 'LineWidth', 2, 'DisplayName', 'Velocita
    Desiderata (Ref)');
17 h_real = animatedline('Color', 'b', 'LineWidth', 2, 'DisplayName', 'Velocita
    Reale da Encoder');
18
19 legend('Location', 'northwest');
20 xlabel('Campioni (1 campione = 10ms)');
21 ylabel('Velocita [m/s]');
22 grid on;
23 ylim([0, 1.5]); % Alzato per supportare velocita fino a 1 m/s e oltre
24
25 somma_errori_quadrati = 0;
26 campioni_validi = 0;
27
28 % 3. Ciclo di Lettura e Plotting
29 disp('Inizio lettura dati... Premi Ctrl+C nella Command Window per fermare.')
    ;
30
31 campioni = 1000;
32 for i = 1:campioni
33     try
34         data_str = readline(s);
35         data_split = split(data_str, ',');
36
37         if length(data_split) == 3 && startsWith(data_split(1), "R")
38
39             v_ref = str2double(data_split(2));
40             v_real = str2double(data_split(3));
41
42             % CALCOLO MINIMI QUADRATI (MSE)
43             errore_attuale = v_ref - v_real;
44             somma_errori_quadrati = somma_errori_quadrati + (errore_attuale
                ^2);
45             campioni_validi = campioni_validi + 1;
46
47             mse_attuale = somma_errori_quadrati / campioni_validi;
48
49             title(sprintf('Inseguimento Velocita | MSE: %.6f', mse_attuale));
50
51             addpoints(h_ref, campioni_validi, v_ref);
52             addpoints(h_real, campioni_validi, v_real);
53
54             drawnow limitrate;
55         end
56     catch
57         continue;
58     end
59 end
60
61 disp('Acquisizione completata.');
```

Listing 5.2: Codice in MATLAB: controllo trazione

Il codice appena illustrato si occupa di inizializzare la comunicazione seriale configurando la porta dedicata e il corretto baud rate. Successivamente, predispone un grafico dinamico e,

tramite un ciclo iterativo, legge i pacchetti in arrivo isolando i valori della velocità di riferimento e di quella reale. Oltre a tracciare le due curve in tempo reale, lo script calcola e aggiorna dinamicamente a schermo il valore dell'Errore Quadratico Medio (MSE), permettendo di valutare istantaneamente la bontà dell'inseguimento del setpoint, per poi arrestarsi automaticamente al raggiungimento del limite prefissato di campioni. La taratura del controllore è stata ottimizzata specificamente per una velocità di riferimento di 1 m/s, di cui è stata analizzata la risposta a gradino. Il grafico ottenuto è il seguente:

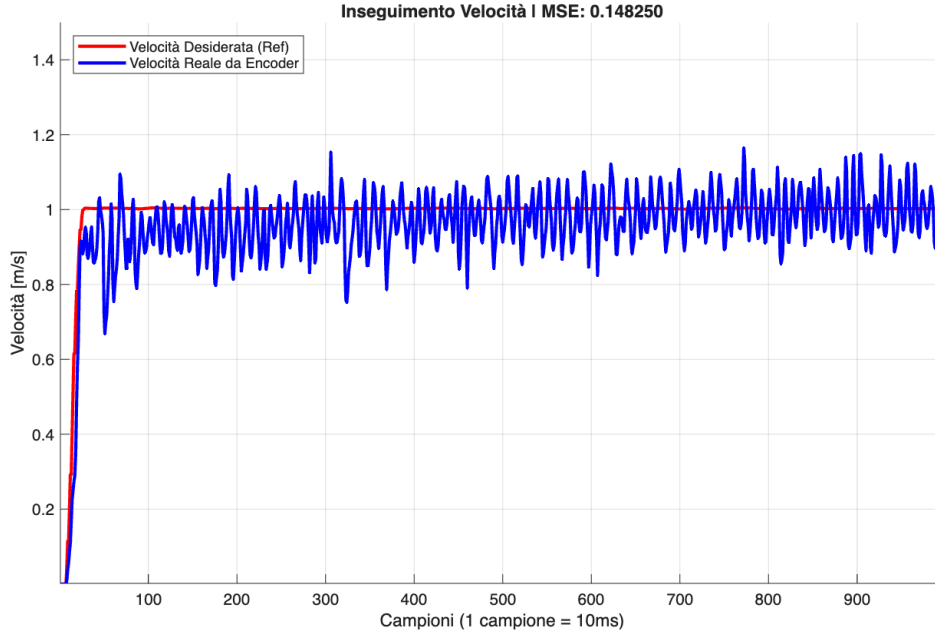


Figure 5.1: *Risposta a gradino del controllo di trazione a 1.0 m/s.*

Dal grafico è possibile osservare il confronto tra la velocità desiderata (curva rossa) e la velocità reale misurata tramite l'encoder (curva blu). Il sistema mostra un tempo di salita (*rise time*) estremamente reattivo: il veicolo raggiunge il valore di regime in circa 15-20 campioni, corrispondenti a soli 150-200 ms. Una volta raggiunto il *setpoint*, il valore medio della velocità reale si allinea in modo accurato al riferimento. Sono tuttavia evidenti delle fluttuazioni ad alta frequenza attorno al valore nominale, tali oscillazioni non sono sintomo di instabilità del controllore, ma sono fisiologicamente imputabili al rumore intrinseco nella derivazione numerica della posizione dell'encoder per il calcolo della velocità, unito alle naturali tolleranze e ai giochi meccanici della trasmissione del veicolo. Le prestazioni complessive di inseguimento sono quantificate dall'Errore Quadratico Medio (MSE), che per questo test risulta contenuto a un valore di 0.148250.

Infine, per verificare la robustezza e la stabilità del regolatore al variare delle condizioni operative, il PID tarato sul riferimento di 1 m/s è stato collaudato anche a velocità inferiori. Nello specifico, sono stati condotti test a velocità di 0.9, 0.8, 0.6 e 0.4 m/s.

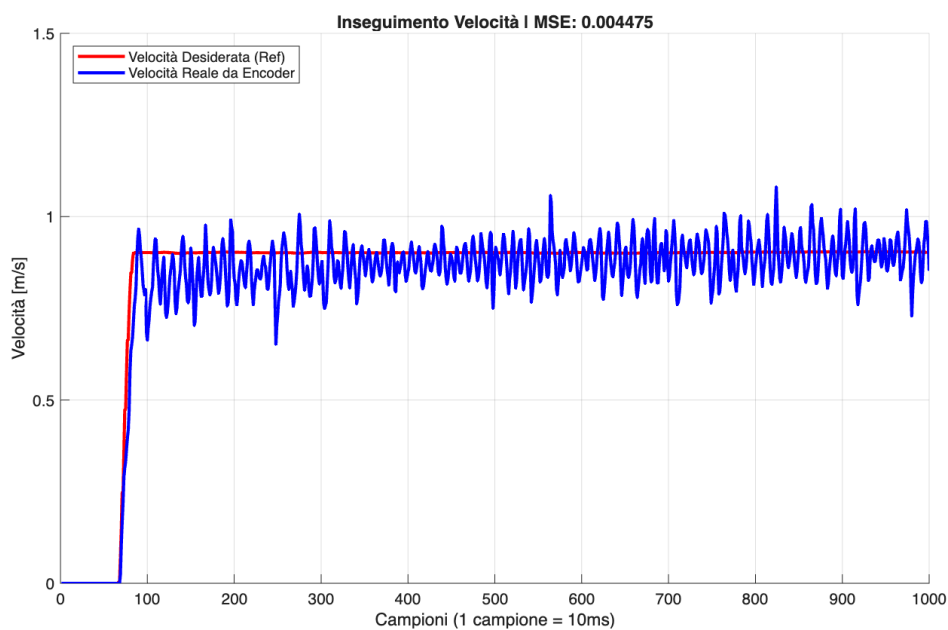


Figure 5.2: *Risposta a gradino del controllo di trazione a 0.9 m/s*

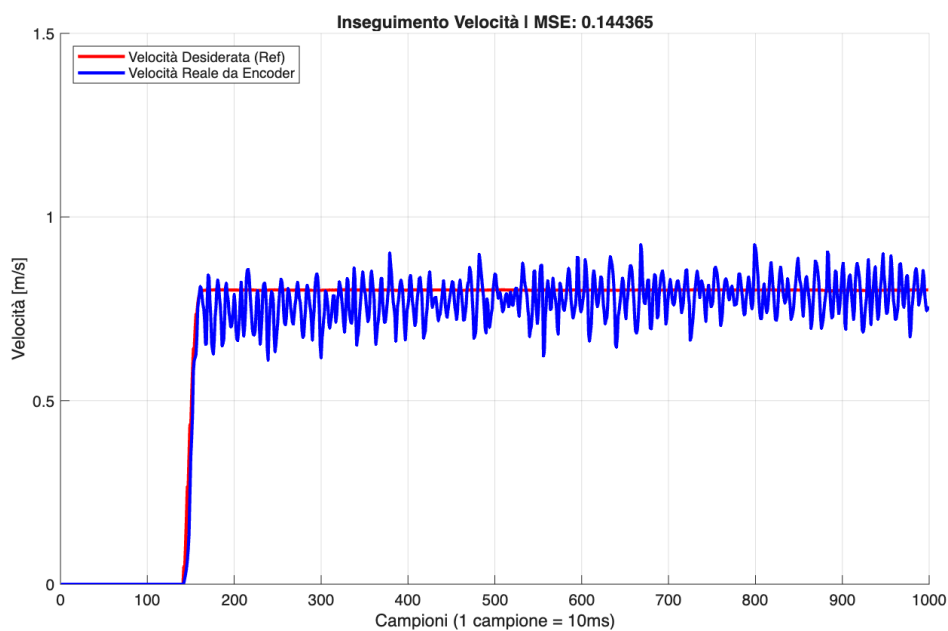


Figure 5.3: *Risposta a gradino del controllo di trazione a 0.8 m/s*

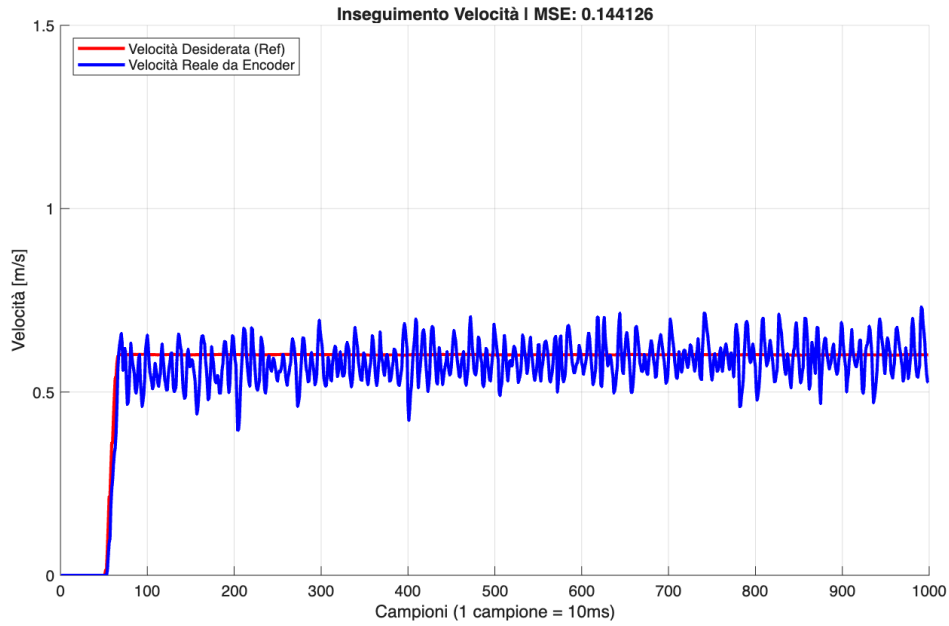


Figure 5.4: *Risposta a gradino del controllo di trazione a 0.6 m/s*

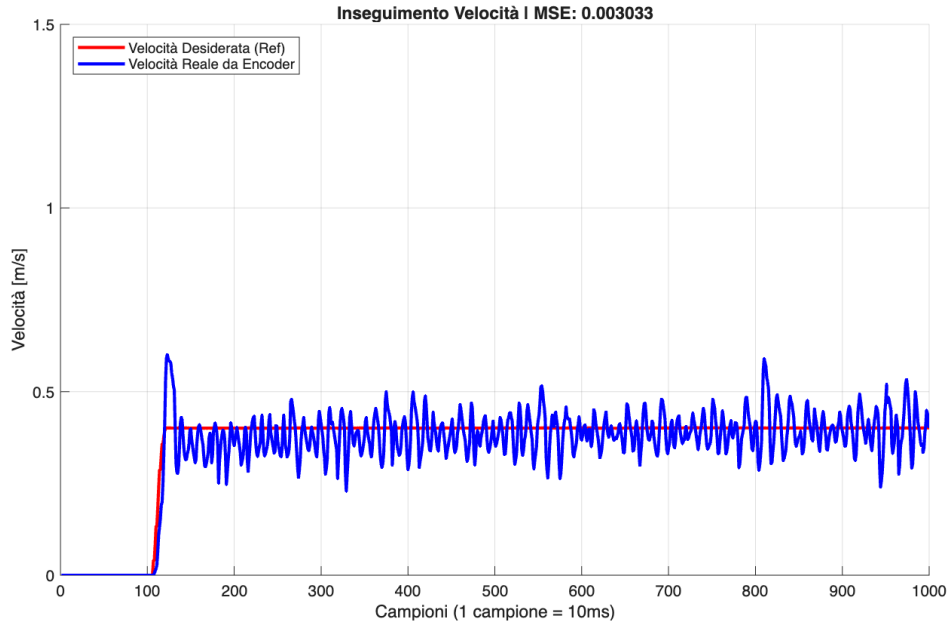


Figure 5.5: *Risposta a gradino del controllo di trazione a 0.4 m/s*

L'analisi dei grafici relativi ai test a velocità decrescenti conferma un comportamento coerente e robusto del controllore PI. In tutte le prove, il tempo di salita (*rise time*) si mantiene pressoché costante e rapido, garantendo l'inseguimento del riferimento in circa 150-200 ms senza evidenziare sovraelongazioni critiche. A regime, il valore medio della velocità reale si attesta fedelmente sul setpoint desiderato, dimostrando l'efficacia dell'azione integrale nell'annullare l'errore statico. È possibile notare come le oscillazioni ad alta frequenza mantengano un'am-

piezza paragonabile e costante in tutti i regimi di marcia analizzati, tale invarianza avvalorava l'ipotesi che le fluttuazioni non siano imputabili a un'instabilità dell'algoritmo di controllo, bensì al rumore di quantizzazione intrinseco nella lettura dell'encoder e alle tolleranze meccaniche della trasmissione. I valori dell'Errore Quadratico Medio (MSE), registrati in tempo reale durante i test, si mantengono contenuti in ogni scenario, confermando la validità e la scalabilità della taratura effettuata alla velocità nominale di 1 m/s.

5.2 Sterzo

Per quanto riguarda il controllo direzionale dello sterzo, la sintesi del controllore e la taratura dei parametri sono state condotte per via sperimentale. Nello specifico, i test per il dimensionamento del regolatore PI sono stati eseguiti mantenendo il veicolo a una velocità longitudinale costante di 1 m/s e imponendo un riferimento per la velocità di imbardata (yaw rate) pari a 1 rad/s. Le costanti di guadagno ottimali individuate, il tempo di campionamento dell'anello di controllo e i limiti di saturazione dell'azione di controllo (impostati tra -20° e $+20^\circ$ per rispettare i vincoli meccanici del servomotore ed evitare forzature strutturali) sono stati definiti all'interno del firmware come segue:

```

1 //STEERING PID
2 #define STEERING_SAMPLING_TIME 0.01 //[s]
3 #define MAX_U_STEERING 20
4 #define MIN_U_STEERING -20
5 #define KP_STEERING 120
6 #define KI_STEERING 60

```

Listing 5.3: "Configuration.h"

Per la validazione visiva e l'estrapolazione dei risultati, i dati telemetrici sono stati acquisiti in tempo reale via seriale, salvati all'interno di un file di testo (.txt) e infine rielaborati utilizzando il seguente script MATLAB:

```

1 % 1. Legge i dati dal file
2 dati = readmatrix(['sterzo120_60_FIR3_IMU']);
3
4 % 2. Separa le colonne
5 setpoint = dati(:, 1); % Prima colonna: comando dal radiocomando
6 imu_reale = dati(:, 2); % Seconda colonna: la realta letta dal sensore BN0055
7
8 % 3. Crea l'asse del tempo
9 tempo = (0:length(setpoint)-1) * 0.01;
10
11 % 4. Disegna il grafico
12 figure;
13 plot(tempo, setpoint, 'r--', 'LineWidth', 2); hold on;
14 plot(tempo, imu_reale, 'b-', 'LineWidth', 2);
15
16 % 5. Plot
17 grid on;
18 title('Analisi Risposta Dinamica FIR IMU: Setpoint vs IMU');
19 xlabel('Tempo (secondi)');
20 ylabel('Yaw Rate (rad/s)');
21 legend('Comando (Riferimento)', 'Realta (IMU)', 'Location', 'best');

```

Listing 5.4: Codice in MATLAB: controllo sterzo

Un aspetto cruciale nell'implementazione di questo anello di controllo è stato il filtraggio del segnale letto dal sensore inerziale (IMU), intrinsecamente affetto da rumore ad alta frequenza. Per ottimizzare la pulizia del segnale, sono state condotte due diverse prove sperimentali sul filtro digitale: la prima configurazione prevedeva l'utilizzo di un doppio filtro FIR posto in cascata sulla linea di retroazione, mentre la seconda impiegava un unico filtro FIR.

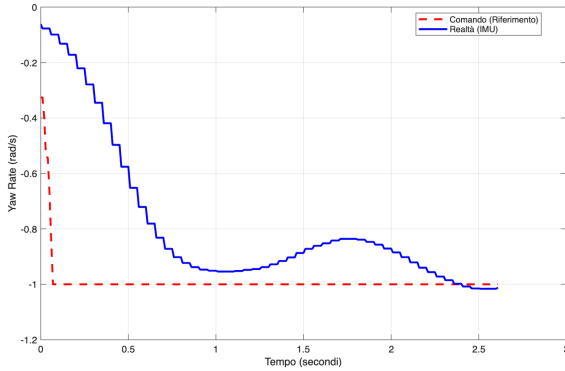


Figure 5.6: *FIR a cascata in retroazione*

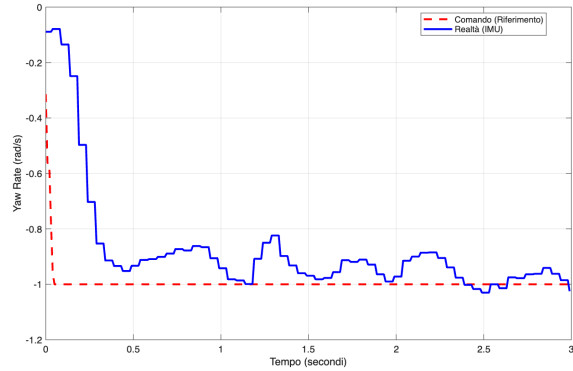


Figure 5.7: *FIR in retroazione*

Nel primo grafico, relativo all'utilizzo del doppio filtro in cascata, si può notare come il segnale misurato (curva blu) risulti fluido. Tuttavia, questo eccessivo filtraggio introduce un grave ritardo di fase nell'anello di controllo. A causa di questo ritardo, il controllore reagisce in ritardo all'errore, innescando un'ampia e lenta oscillazione sottosmorzata. Nel secondo grafico, che illustra la risposta con il singolo filtro FIR, il comportamento dinamico risulta nettamente superiore. Il tempo di risposta del veicolo migliora e, inoltre, una volta giunto a regime, il sistema oscilla con frequenza maggiore ma con ampiezza estremamente ridotta attorno al riferimento desiderato, senza presentare le profonde derive osservate nel test precedente. L'utilizzo di un singolo filtro FIR in retroazione si è dunque confermata la scelta decisiva. A valle della scelta architetturale relativa all'utilizzo di un singolo filtro FIR in

retroazione, si è proceduto all'ottimizzazione del suo ordine, ovvero l'ampiezza della finestra mobile di campionamento. L'obiettivo era individuare la dimensione minima capace di garantire una pulizia del segnale sufficientemente precisa, riducendo al minimo indispensabile il ritardo di elaborazione introdotto nell'anello di controllo. L'implementazione software del

filtro, basata su un array adibito a buffer circolare, è stata definita come segue:

```
1 //FIR_
2 #define FIR_ORDER 3 // Finestra di 3 campioni
3 float fir_buffer[FIR_ORDER] = {0}; // L'array per il buffer del FIR
4 int fir_index = 0;
```

Listing 5.5: "Main.c"

Inizialmente, l'ordine del filtro è stato impostato a 3 campioni. Al fine di validare tale parametro e cercare il miglior compromesso prestazionale, sono stati eseguiti ulteriori test

empirici variando l'ampiezza della finestra a 6, 4 e, infine, a 2 campioni. Nelle figure seguenti sono riportati i grafici della telemetria relativi a ciascuna delle finestre testate.

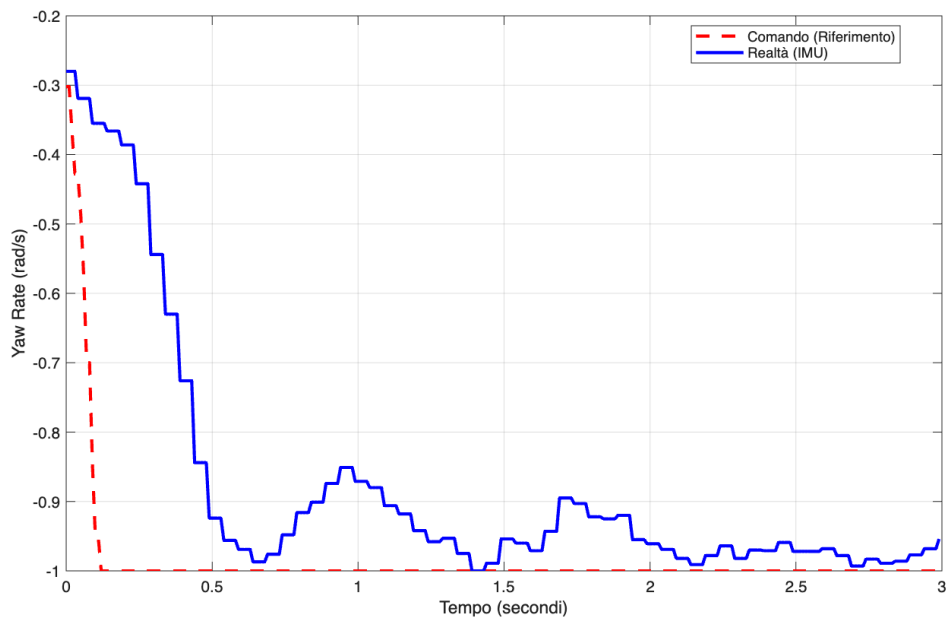


Figure 5.8: *Finestra di 6 campioni*

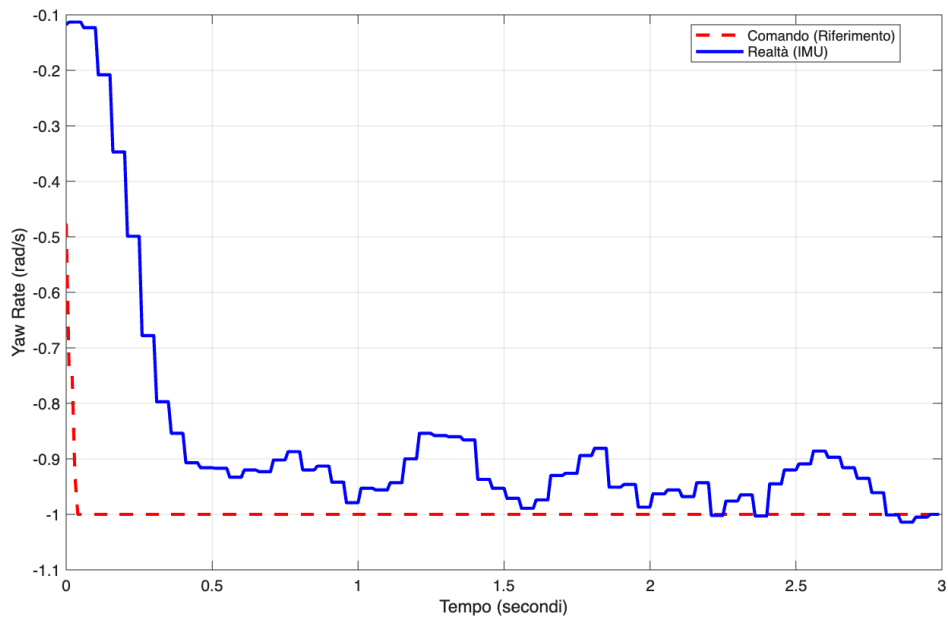


Figure 5.9: *Finestra di 4 campioni*

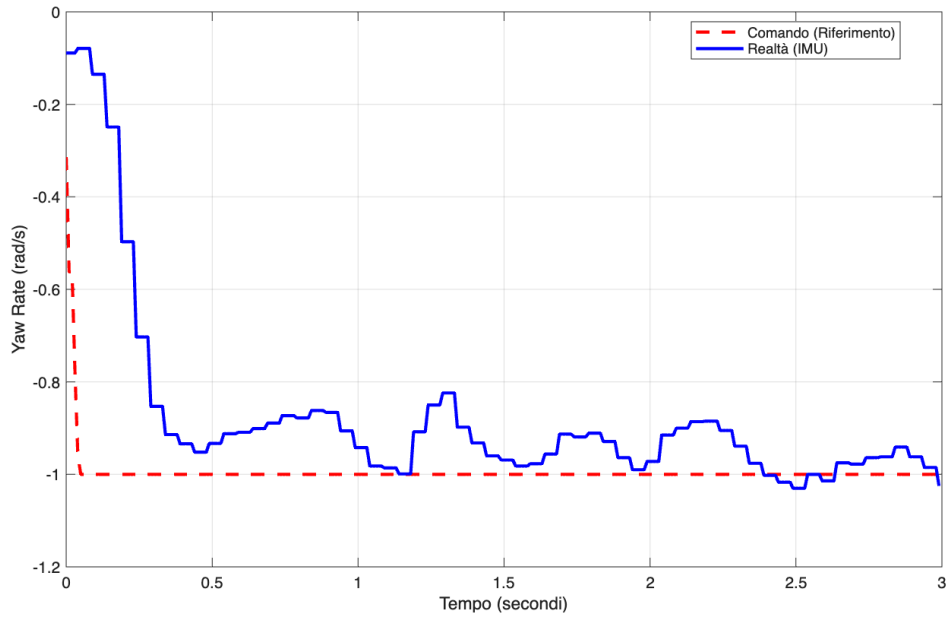


Figure 5.10: *Finestra di 3 campioni*

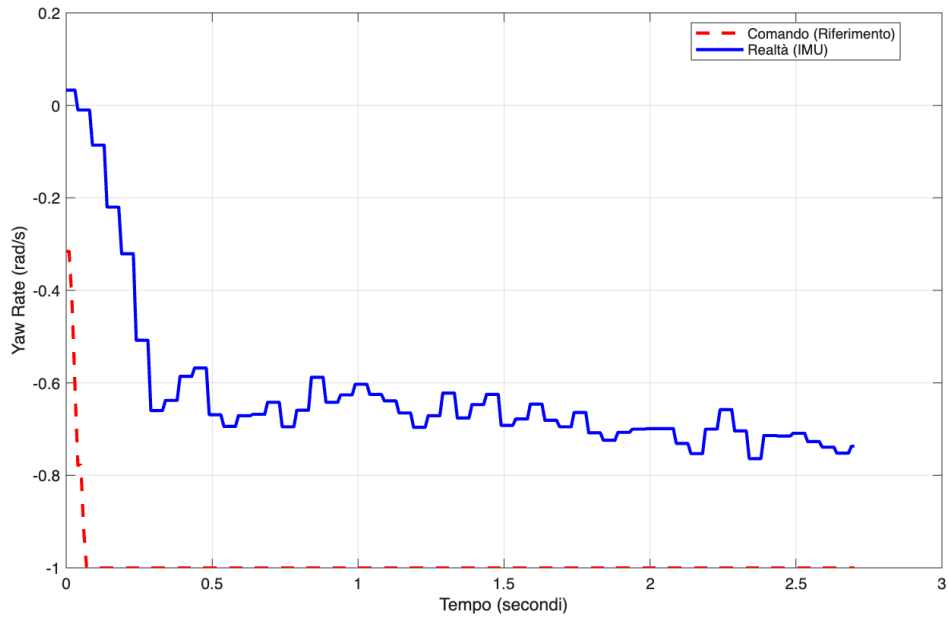


Figure 5.11: *Finestra di 2 campioni*

Nel primo grafico, corrispondente a una finestra di 6 campioni, si evince l'effetto di un filtraggio troppo aggressivo: sebbene la curva reale (in blu) risulti molto liscia, il notevole ritardo di fase introdotto dal buffer fa sì che il veicolo impieghi oltre un secondo per raggiungere la prima volta il riferimento di -1 rad/s. Riducendo la finestra a 4 campioni, come mostrato nel secondo grafico, il ritardo si attenua e il tempo di salita migliora sensibilmente, ma permane una certa inerzia nel raggiungere e stabilizzare in modo netto il setpoint. Nel quarto grafico,

con il comportamento del sistema con un filtro ridotto a soli 2 campioni, il problema si inverte: la finestra temporale è troppo ristretta e risulta insufficiente per abbattere il rumore ad alta frequenza intrinseco dell'IMU. Il terzo grafico, relativo alla finestra a 3 campioni, conferma in modo evidente la bontà di questa configurazione, che si attesta come il compromesso ideale. Il rumore del sensore viene attenuato quanto basta per garantire un calcolo preciso e stabile dell'errore da parte del controllore, mantenendo al contempo un ritardo di elaborazione del tutto trascurabile. Una volta consolidata l'architettura di controllo con il singolo filtro FIR

in retroazione impostato su una finestra di 3 campioni, l'analisi sperimentale è proseguita per valutare la robustezza e l'adattabilità del regolatore al variare delle condizioni cinematiche. Le prove successive sono state quindi suddivise in due fasi distinte: la variazione della velocità di traslazione e la variazione del riferimento di imbardata, yaw rate.

5.2.1 Variazione velocità lineare

Per quanto riguarda la prima fase, l'obiettivo è stato quello di osservare come il controllo direzionale reagisse a velocità di crociera differenti rispetto a quella nominale di 1 m/s utilizzata per la taratura iniziale. A tale scopo, sono stati eseguiti test specifici impostando la trazione a 0.8 m/s e, successivamente, a 0.9 m/s.

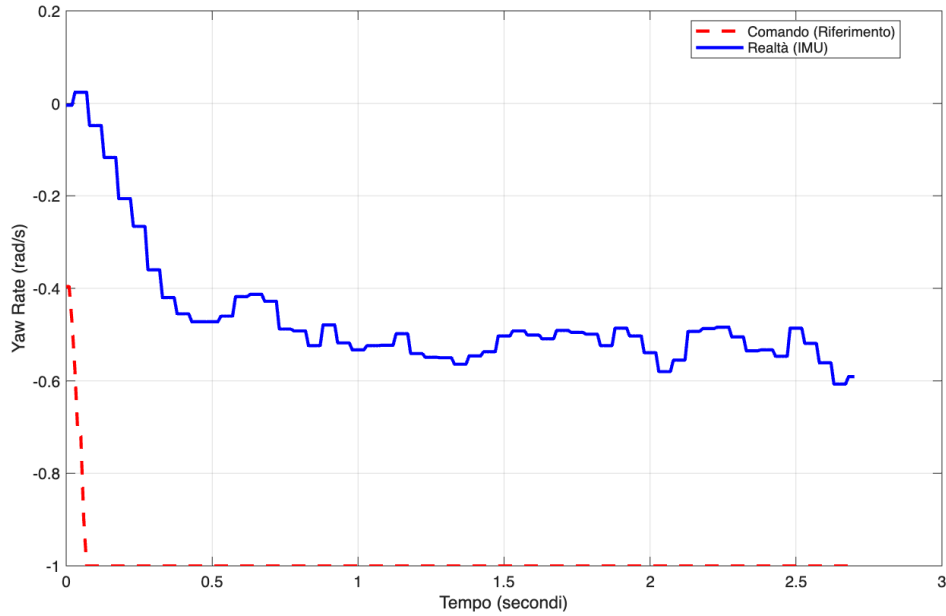


Figure 5.12: *Risposta a gradino del controllo di trazione a 0.9 m/s*

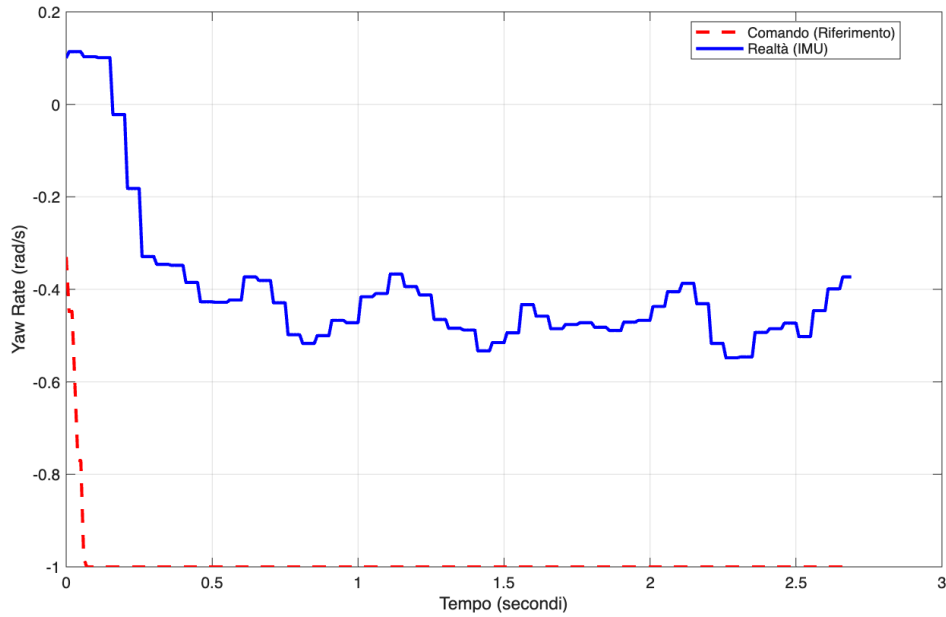


Figure 5.13: *Risposta a gradino del controllo di trazione a 0.8 m/s*

Come si evince dai due grafici, la velocità angolare reale (curva blu) non riesce in alcun modo a raggiungere il setpoint desiderato, assestandosi su valori medi nettamente inferiori: circa -0.55 rad/s nel test a 0.9 m/s e circa -0.45 rad/s in quello a 0.8 m/s. Essendo l'angolo di sterzo vincolato via software e hardware a un massimo di $\pm 20^\circ$, il prototipo possiede un raggio di curvatura minimo invalicabile. Poiché la velocità di imbardata è direttamente proporzionale alla velocità longitudinale di marcia, a parità di angolo di sterzo massimo, una riduzione della velocità rettilinea comporta inevitabilmente una diminuzione della massima velocità angolare raggiungibile. In questi scenari, il regolatore riconosce l'errore e satura correttamente l'azione di controllo al suo limite massimo, ma il veicolo risulta fisicamente impossibilitato a curvare in modo più restrittivo.

5.2.2 Variazione velocità angolare

Nella seconda fase di test, riportando e mantenendo costante la velocità di traslazione al valore nominale, è stata sollecitata la dinamica laterale variando il setpoint dello yaw rate. Il sistema è stato sottoposto a richieste di sterzata di diversa entità: sono state investigate dinamiche più blande, testando riferimenti di 0.6 rad/s e 0.8 rad/s, e manovre via via più aggressive, innalzando la richiesta a 1.2 rad/s e 1.4 rad/s. Questo ha permesso di delineare in modo completo i limiti operativi del controllore e i vincoli fisici del servomotore.

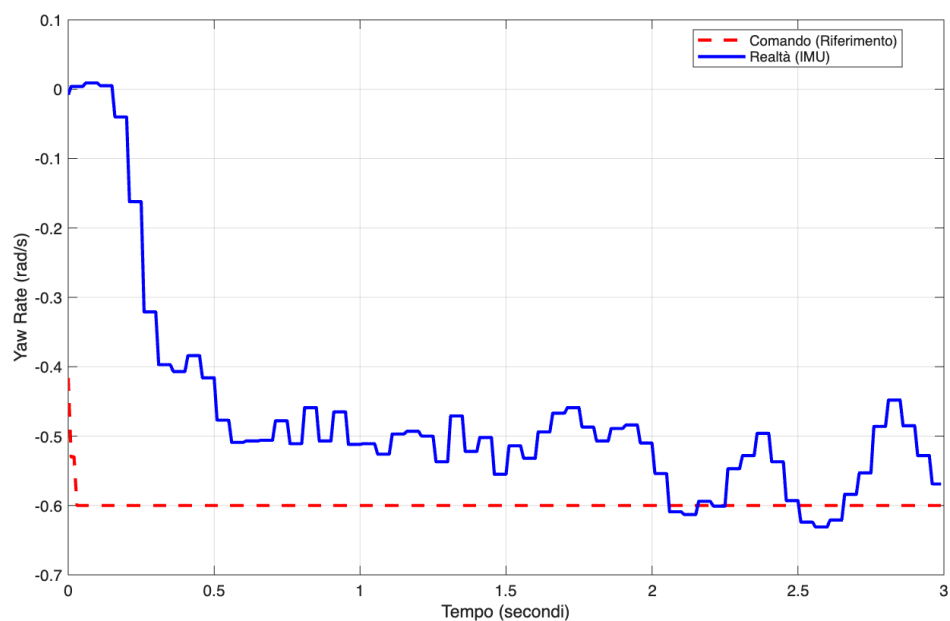


Figure 5.14: *Risposta a gradino del controllo di sterzo, riferimento di -0.6 rad/s.*

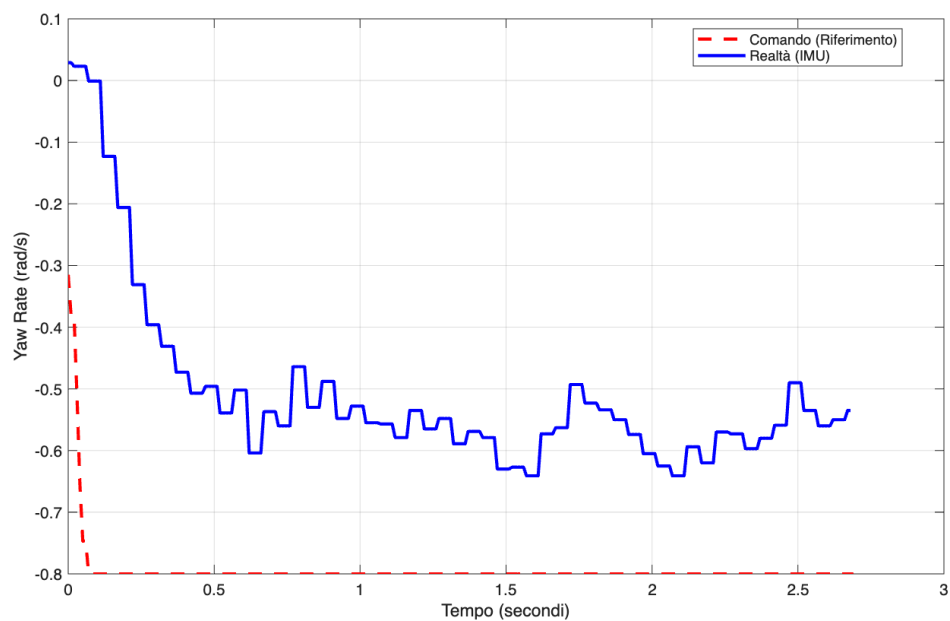


Figure 5.15: *Risposta a gradino del controllo di sterzo, riferimento di -0.8 rad/s.*

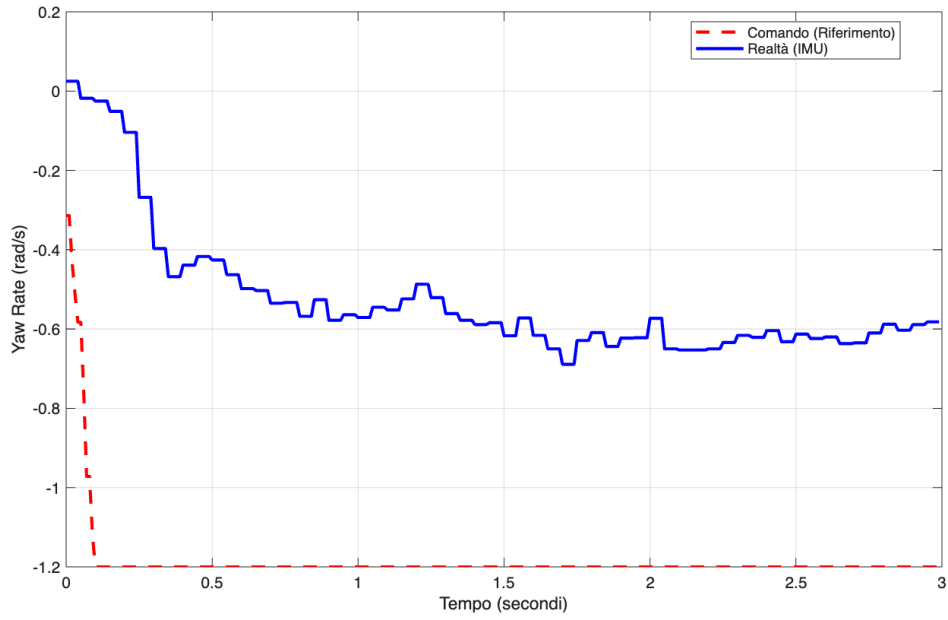


Figure 5.16: *Risposta a gradino del controllo di sterzo, riferimento di -1.2 rad/s.*

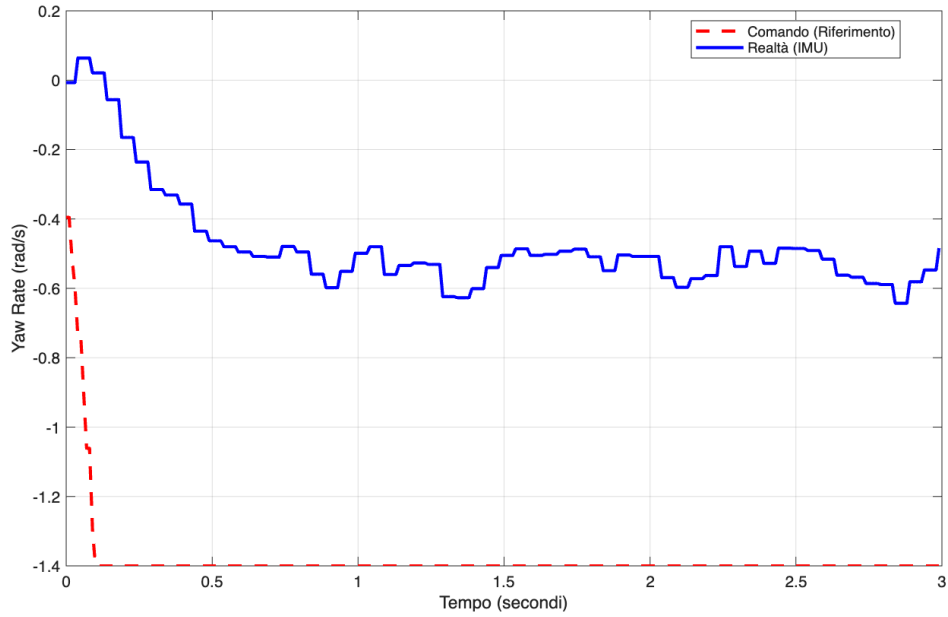


Figure 5.17: *Risposta a gradino del controllo di sterzo, riferimento di -1.4 rad/s.*

Come si evince dalle misurazioni, il riferimento di -0.6 rad/s viene raggiunto e mantenuto dal controllore, rappresentando la soglia massima delle capacità di manovra del prototipo per la velocità di avanzamento in uso. Al contrario, per le richieste superiori -1.2 e -1.4 rad/s, la velocità angolare reale non riesce a inseguire il setpoint, assestandosi e saturando attorno al limite fisico di circa -0.6 rad/s. Avendo raggiunto il raggio di curvatura minimo del telaio,

risulta impossibile generare velocità di imbardata superiori, costringendo il sistema in uno stato di completa saturazione.

Conclusioni e sviluppi futuri

Il task principale consisteva nel realizzare un'architettura di controllo completa, in grado di acquisire i comandi del radiocomando, elaborare i dati sensoriali e tarare adeguatamente i PID per la trazione e lo sterzo. Complessivamente, gli obiettivi fondamentali del progetto sono stati raggiunti. Il controllo di trazione, attuato sul motore DC e retroazionato tramite encoder, ha dimostrato un ottimo inseguimento del riferimento di velocità lineare, evidenziando tempi di salita ridotti e un Errore Quadratico Medio del tutto trascurabile. Parallelamente, grazie all'implementazione di filtri digitali FIR sia singoli che in cascata, che hanno permesso di ripulire efficacemente il segnale, etto dall'IMU, è stato possibile stabilizzare la risposta direzionale del veicolo, ottimizzando le prestazioni dell'anello chiuso senza introdurre latenze eccessive nel sistema. Alla luce dei risultati sperimentali e delle limitazioni fisiche evidenziate, un naturale sviluppo futuro riguarderebbe l'implementazione di diverse mappature di guida, selezionabili in tempo reale tramite i canali ausiliari del radiocomando. Questo approccio permetterebbe di modificare dinamicamente a runtime i guadagni dei regolatori PID, le costanti di tempo dei filtri FIR o i limiti massimi di velocità e imbardata. Si potrebbero così creare profili di guida differenziati, ad esempio una modalità *Eco/Smooth* per transitori dolci e una modalità *Sport* per la massima reattività, adattando istantaneamente il comportamento del prototipo alle esigenze del pilota o alle condizioni della superficie di prova.

Bibliografia

- [1] Microzone, *Microzone 6CH Remote Transmitter Receiver System for Drones/Quadcopters, Modello: MC6C*. Manuale utente e specifiche tecniche (Banda 2.4 GHz, Modulazione FHSS).
- [2] A. Libofsha, “Sviluppo di algoritmi su microcontrollore per il controllo di direzione e velocità per la guida autonoma di un veicolo in scala 1:10.”
- [3] M. Colletta, “Progettazione e sviluppo di algoritmi su dispositivi edge per l’esecuzione di manovre autonome su veicolo in scala 1:10.”
- [4] F. Brunella, “Implementazione di algoritmi di visione per il lane-keeping e il controllo di veicoli autonomi.”
- [5] A. Guagneli, R. Alwan, and L. Ghiddi, “Implementazione di un algoritmo per il controllo di trazione e sterzata di un veicolo in scala 1:10 mediante radiocomando.”
- [6] A. Bonci, “Materiale didattico del corso.” Slide fornite dal docente.